

ECE/CS 757 Advanced Computer Architecture II

Spring 2025

Synchronization

Instructor: Prof. Joshua San Miguel

Some slides adopted from Greg Steffan and Natalie Enright Jerger

Last Lecture

- Loop dependences
- Shared memory and message passing
- Chip multiprocessors
- Parallel applications

This Lecture

- Mutual exclusion
- Test-and-set locks
- Ticket and queuing locks
- Deadlocks

Parallel Programming

Key design decisions:

Extracting parallelism/concurrency

- Pthreads, OpenMP

Communicating data

- Shared memory, message passing

Synchronizing computations

- Locks, barriers

Synchronization

Mutual Exclusion:

Ensures certain operations on shared data (i.e., critical sections) can be performed by only one thread at a time, with no guarantee on ordering.

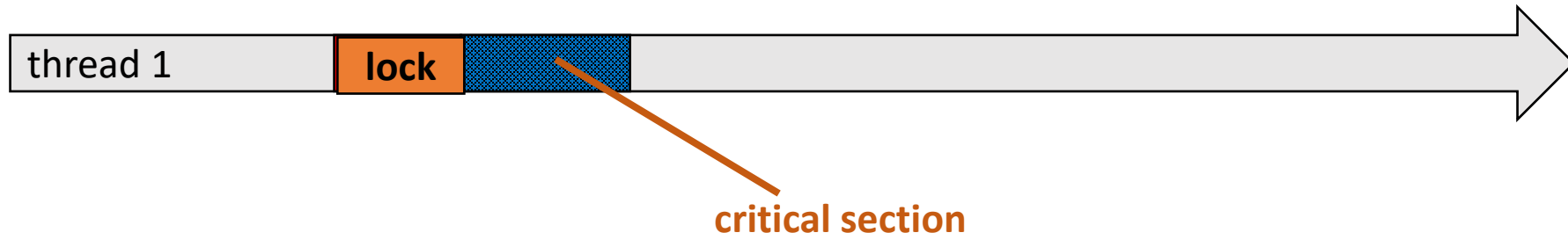
- Locks

Event Synchronization:

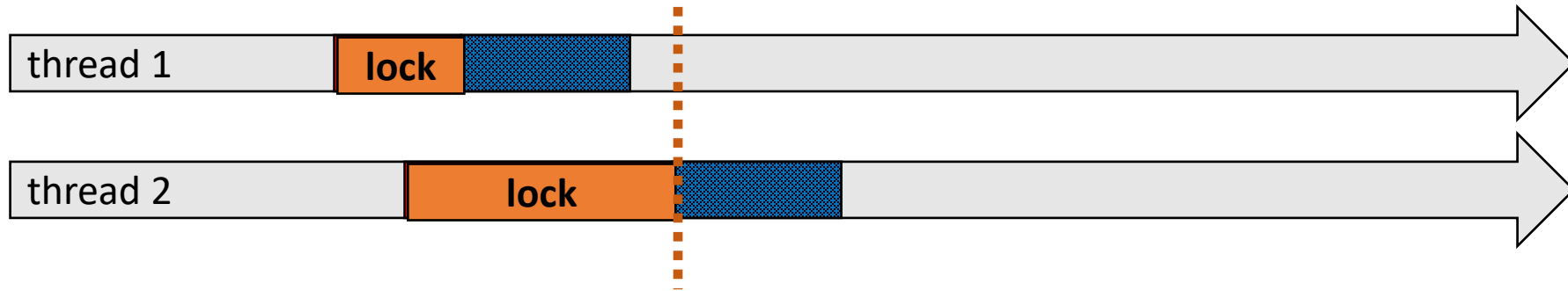
Enforces ordering of events to preserve dependences.

- Barriers
- Producer-consumer (e.g., condition variables, semaphores)

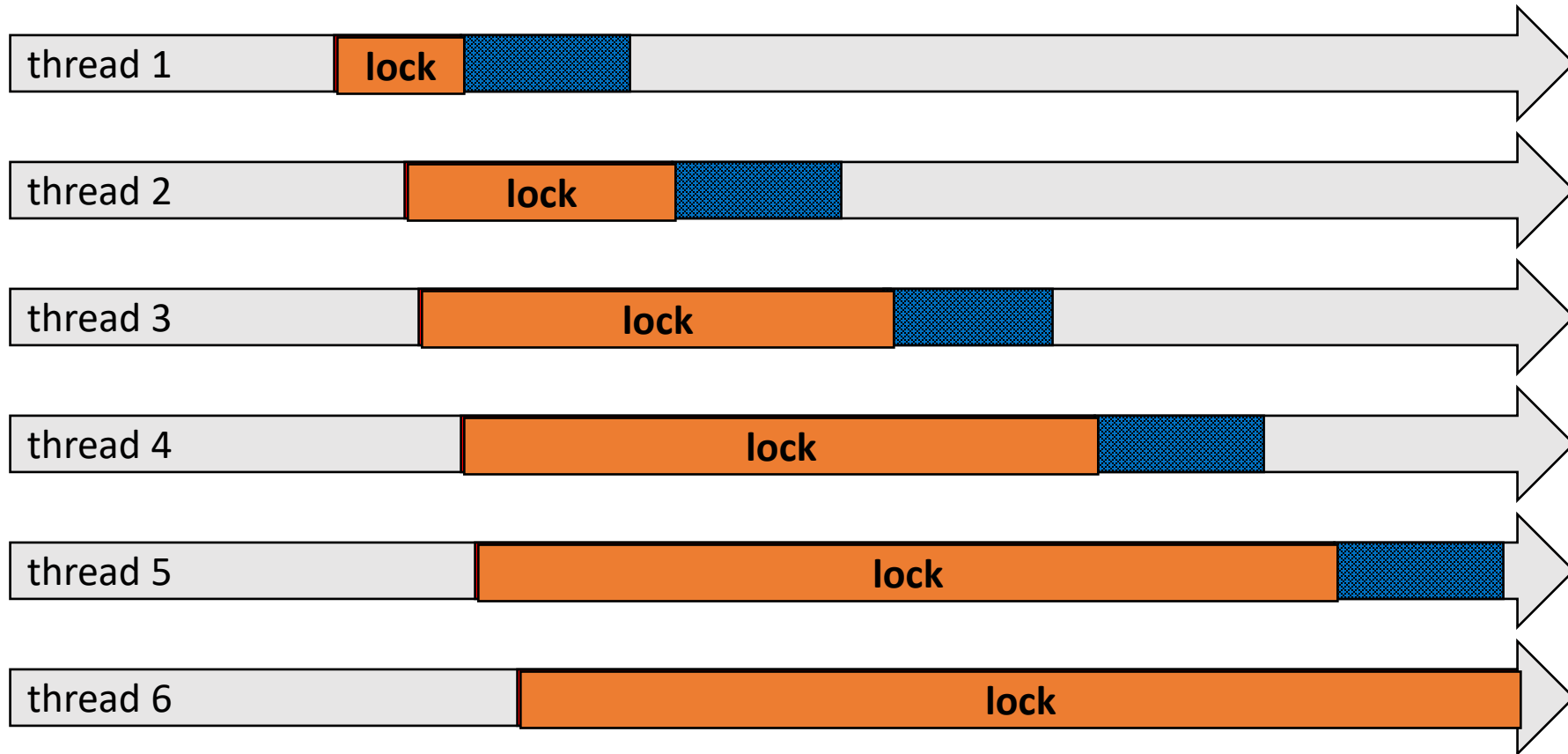
Mutual Exclusion



Mutual Exclusion



Mutual Exclusion



Mutual Exclusion

Major performance bottleneck in multithreaded applications.

Acquire-Release Overhead:

- Implementation cost of locking and unlocking
- Eliminate via caching and locality

Blocking Overhead:

- Fundamental cost of waiting for other threads
- Hide via optimistic conflict detection (e.g., transactional memory)

Locks

Design Objectives:

- Low latency of acquire/release
- Low interconnect traffic
- Low storage cost
- Fairness and starvation avoidance
- Scalability (all of the above in proportion to thread count)

Locks

High-Level (Wrong) Implementation:

LOCK:

```
while(*lockvar == 1);  
*lockvar = 1;
```

UNLOCK:

```
*lockvar = 0;
```

Locks

High-Level (Wrong) Implementation:

LOCK:

```
while(*lockvar == 1);  
*lockvar = 1;
```

context switch!

UNLOCK:

```
*lockvar = 0;
```

Atomic-Read-Write Instructions

Use atomic-read-write instructions:

```
r1 = test-and-set(addr):  
    { r1=*addr; *addr=1; }
```

```
r1 = fetch-and-store(addr, r2):  
    { r1=*addr; *addr=r2; }
```

```
r1 = fetch-and-add(addr, r2):  
    { r1=*addr; *addr=r1+r2; }
```

```
r2 = compare-and-swap(addr, r1, r2):  
    { temp=r2; r2=*addr; if r2==r1 then *addr=temp; }
```



Atomic-Read-Write Instructions

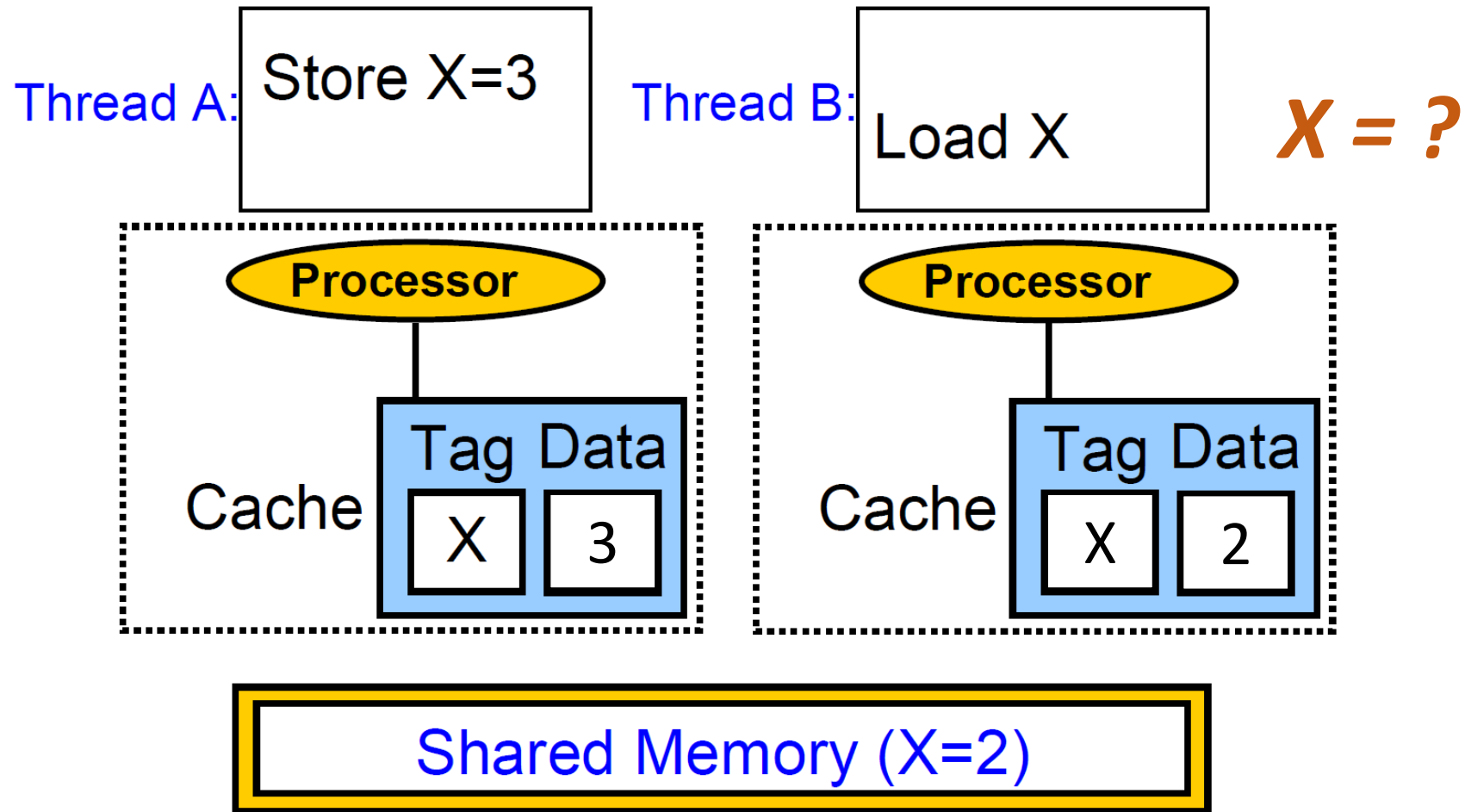
Implementation on bus-based systems:

Hold bus then issue load and store operations without any intervening accesses by other processors.

Implementation on arbitrary interconnects:

Acquire exclusive ownership via cache coherence then perform load and store operations without allowing external coherence requests.

The Cache Coherence Problem



Test-And-Set Lock

LOCK:

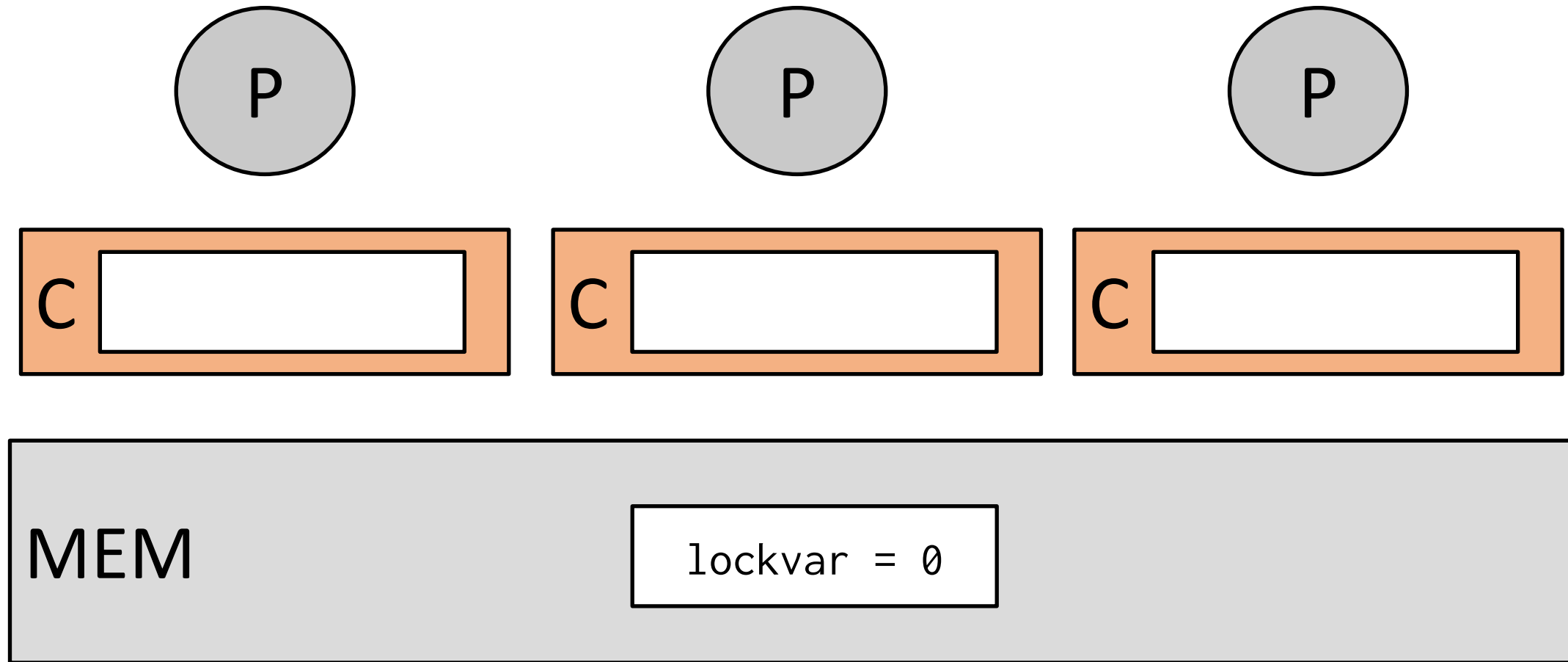
```
while(1) {  
    old = test-and-set(lockvar);  
    if(old == 0) break;  
}
```

UNLOCK:

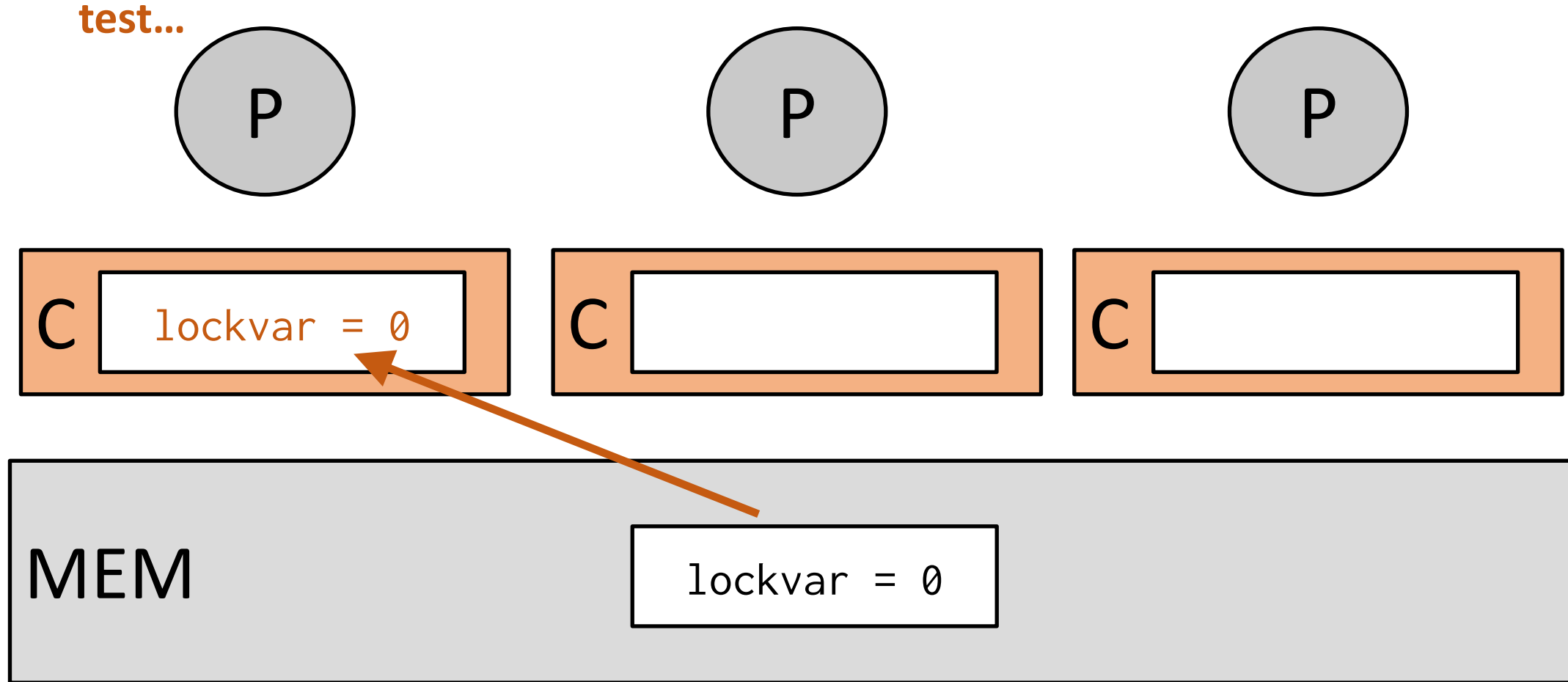
```
*lockvar = 0;
```

What if private data caches?

Test-And-Set Lock

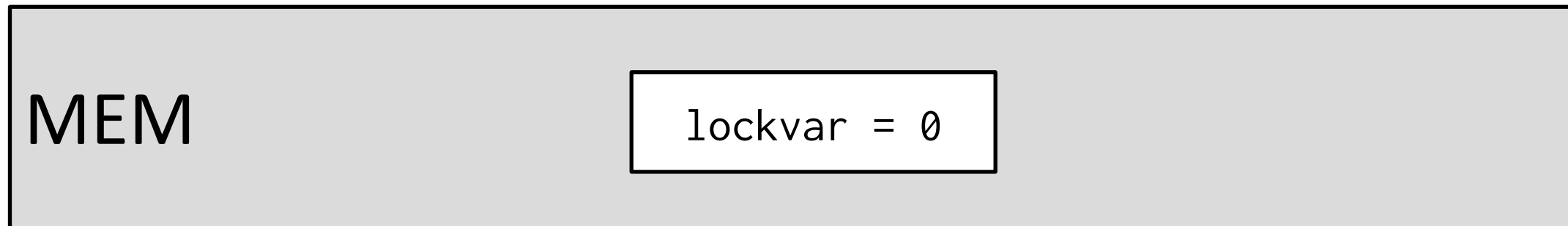
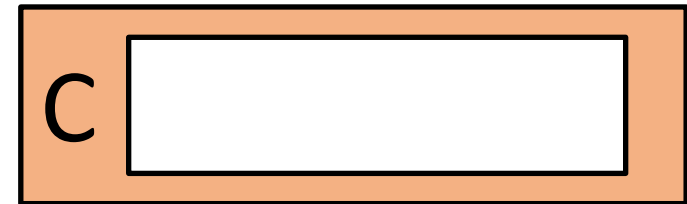
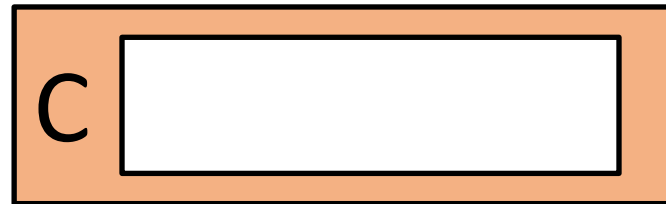
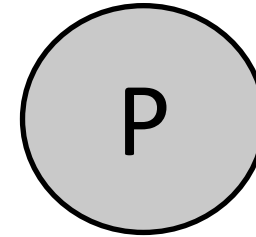
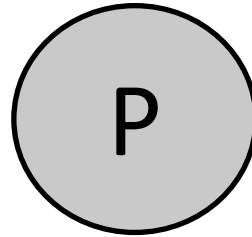
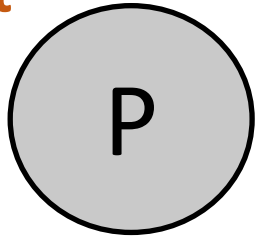


Test-And-Set Lock

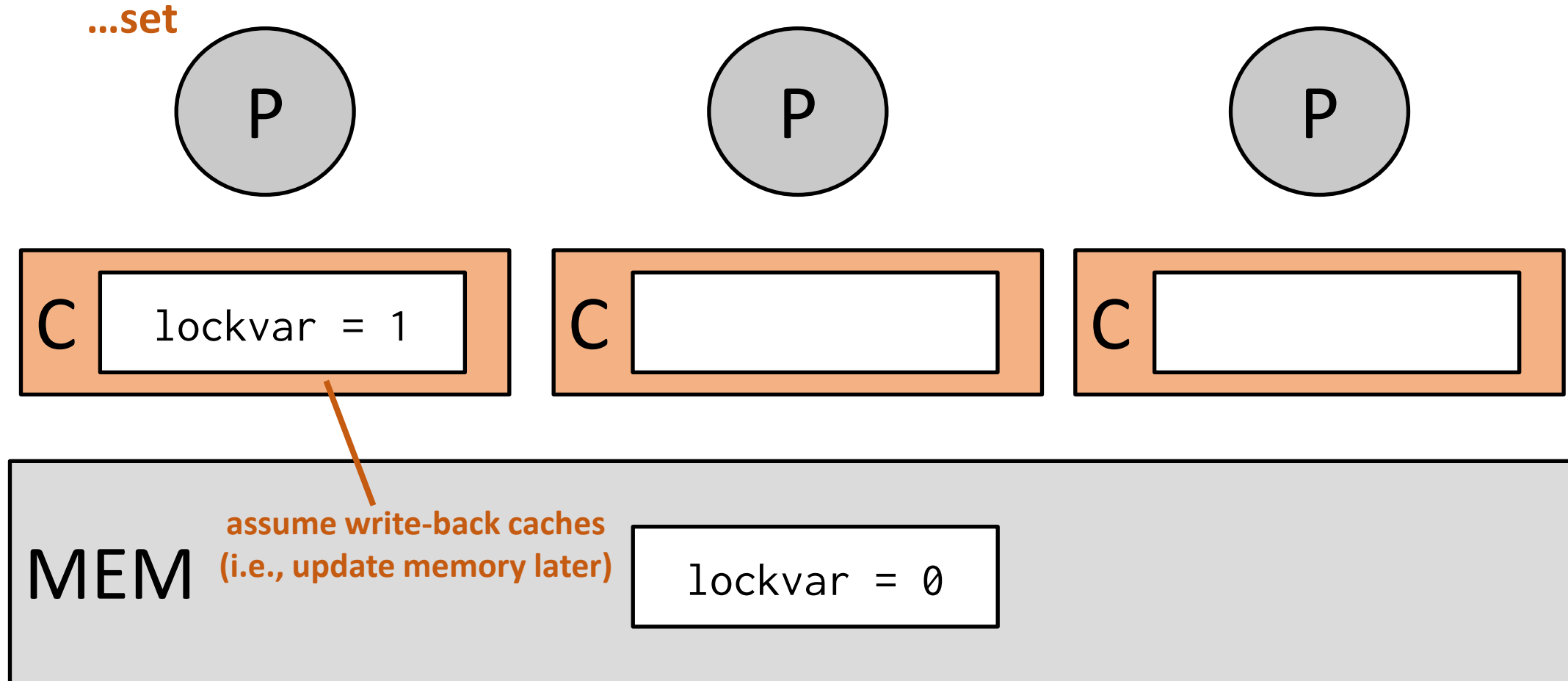


Test-And-Set Lock

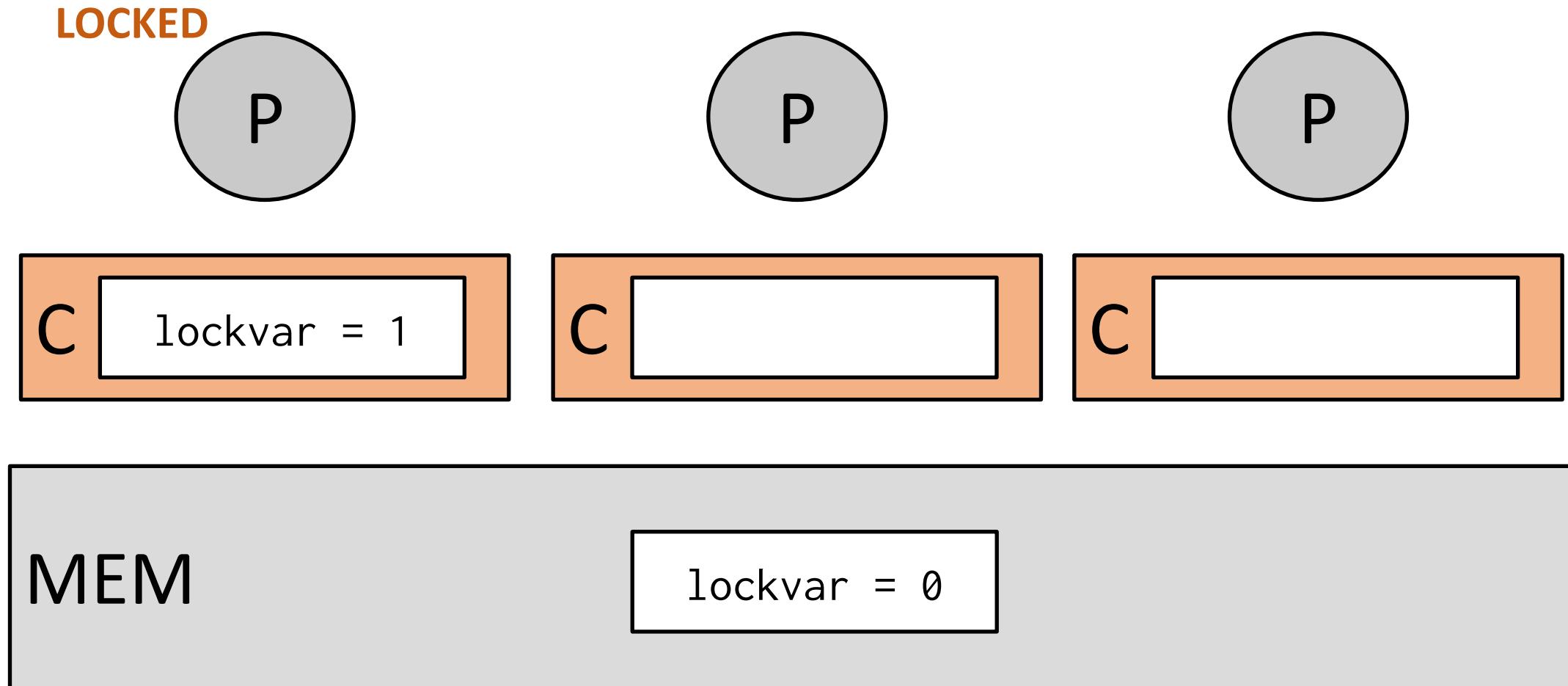
...set



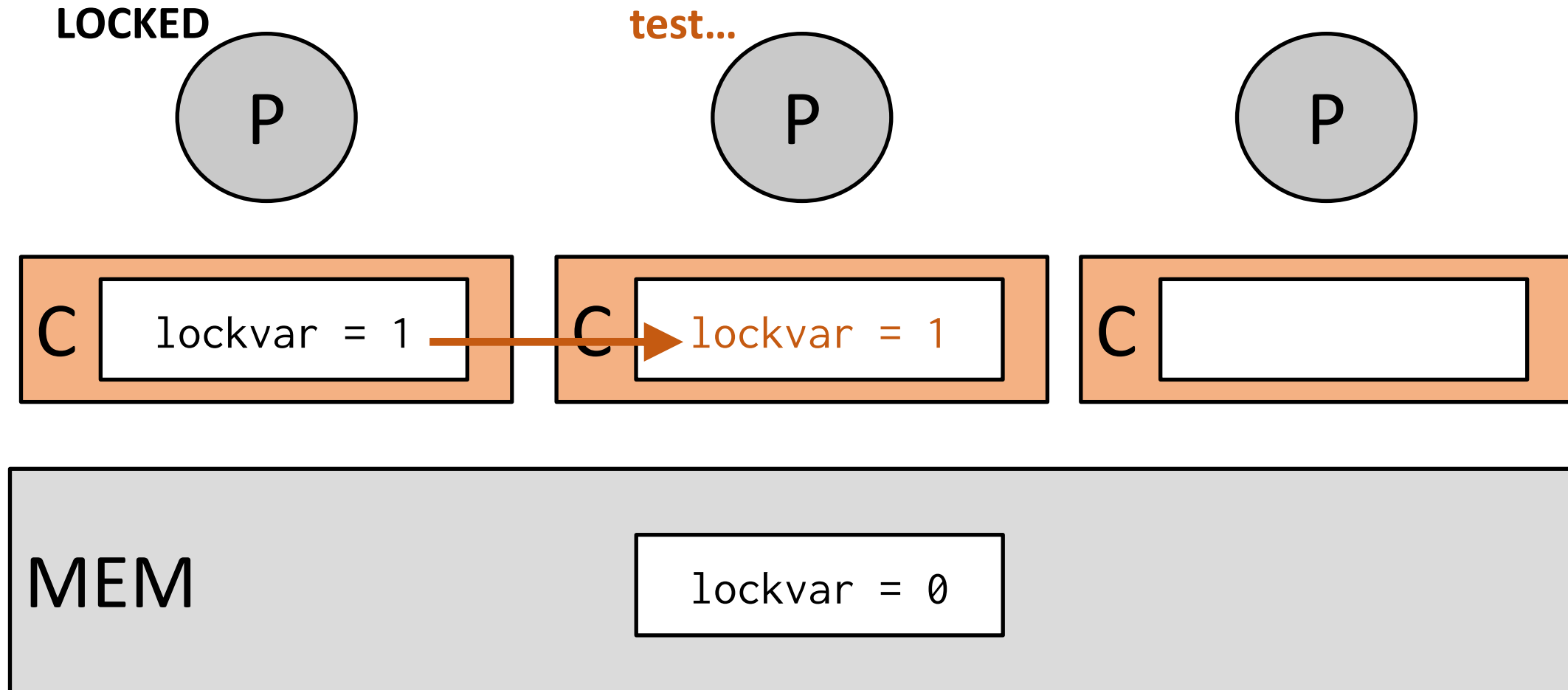
Test-And-Set Lock



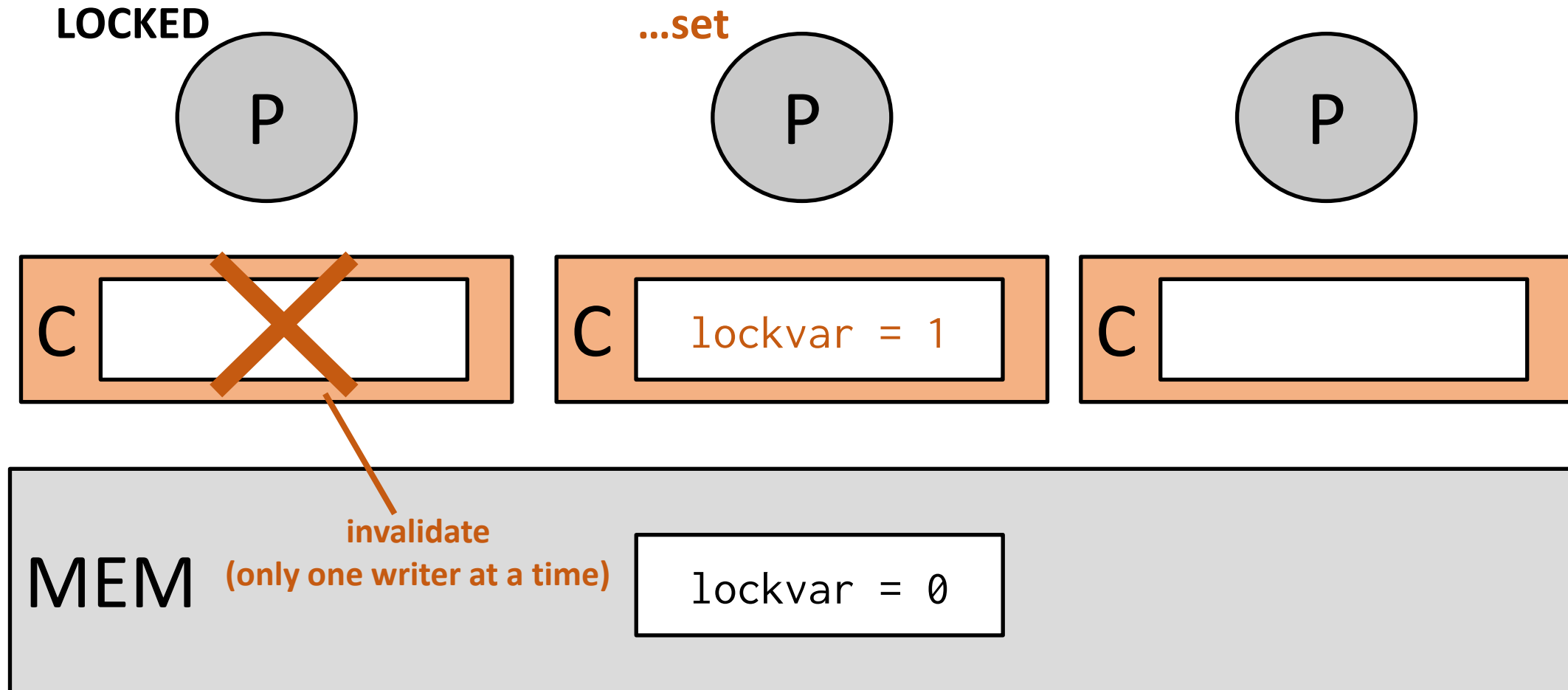
Test-And-Set Lock



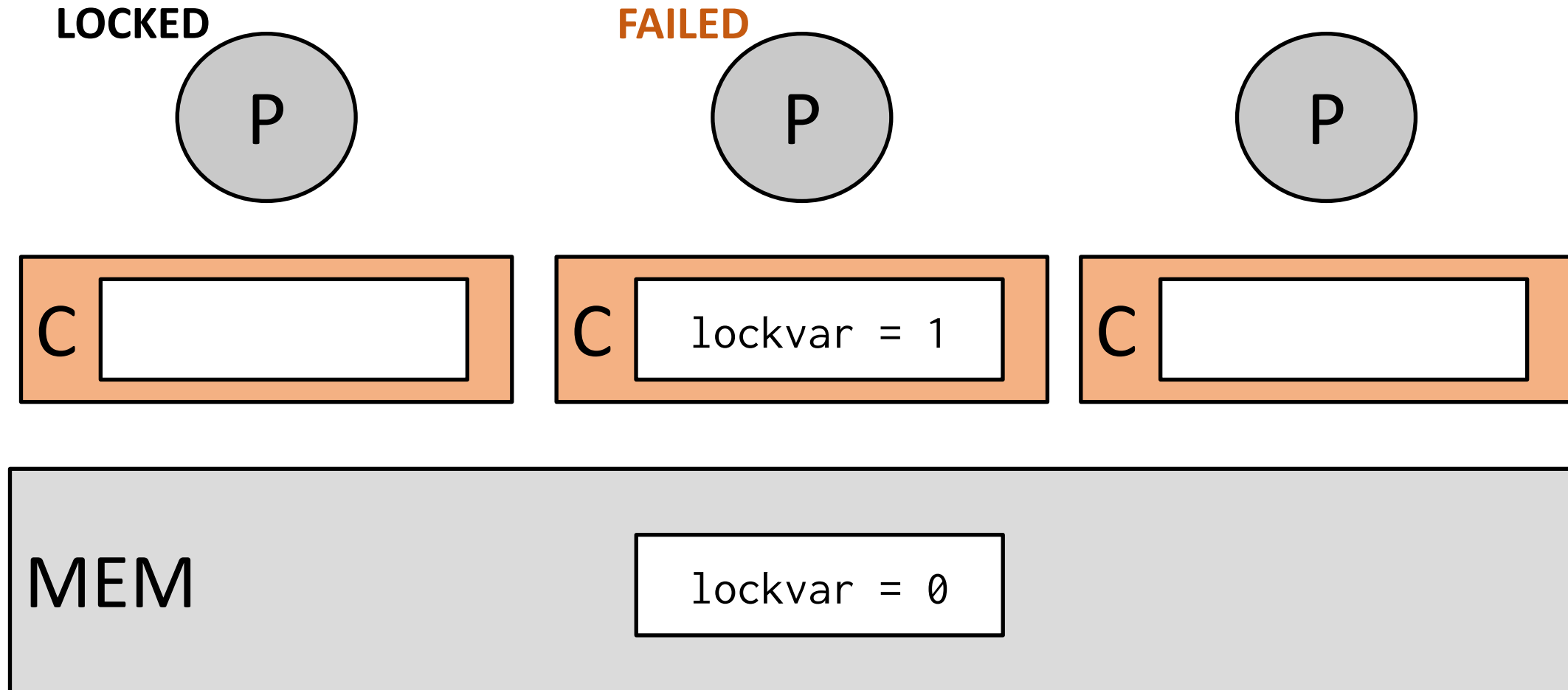
Test-And-Set Lock



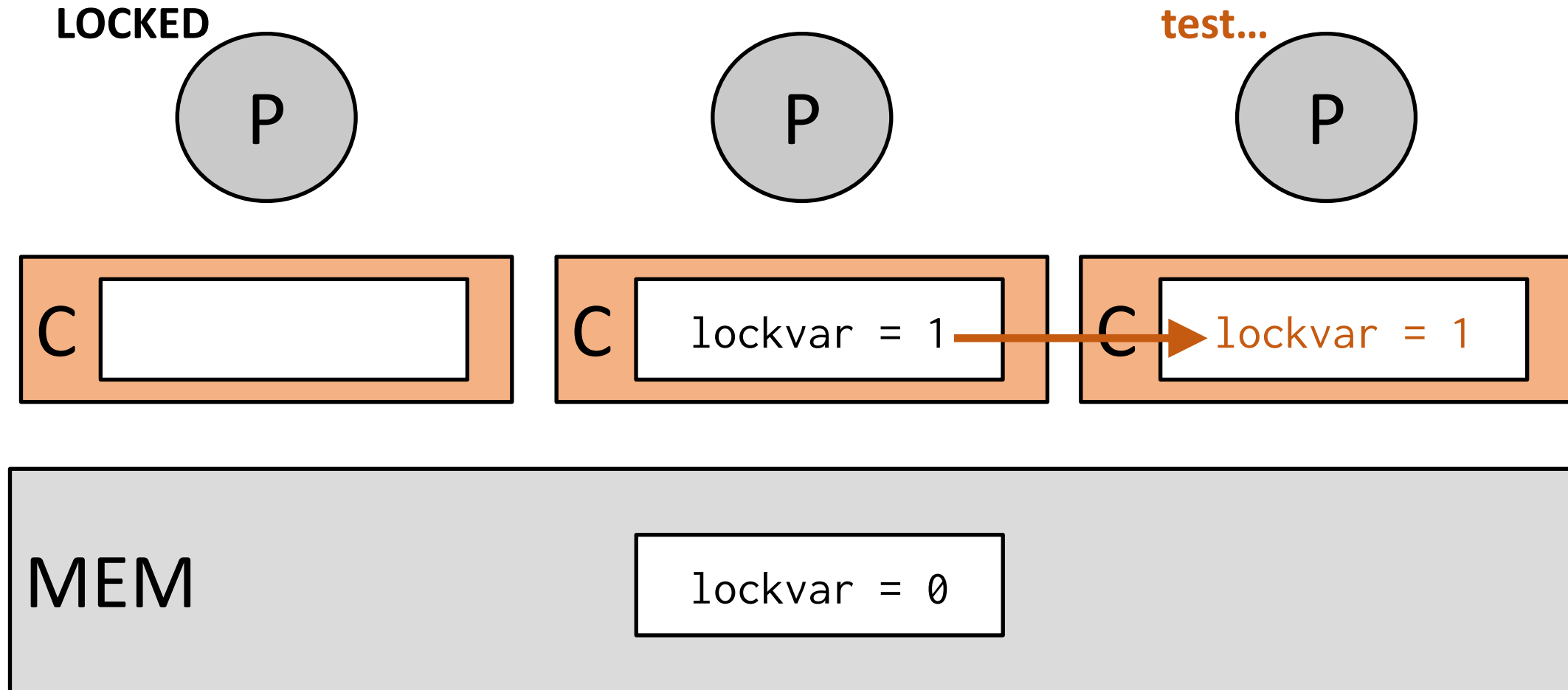
Test-And-Set Lock



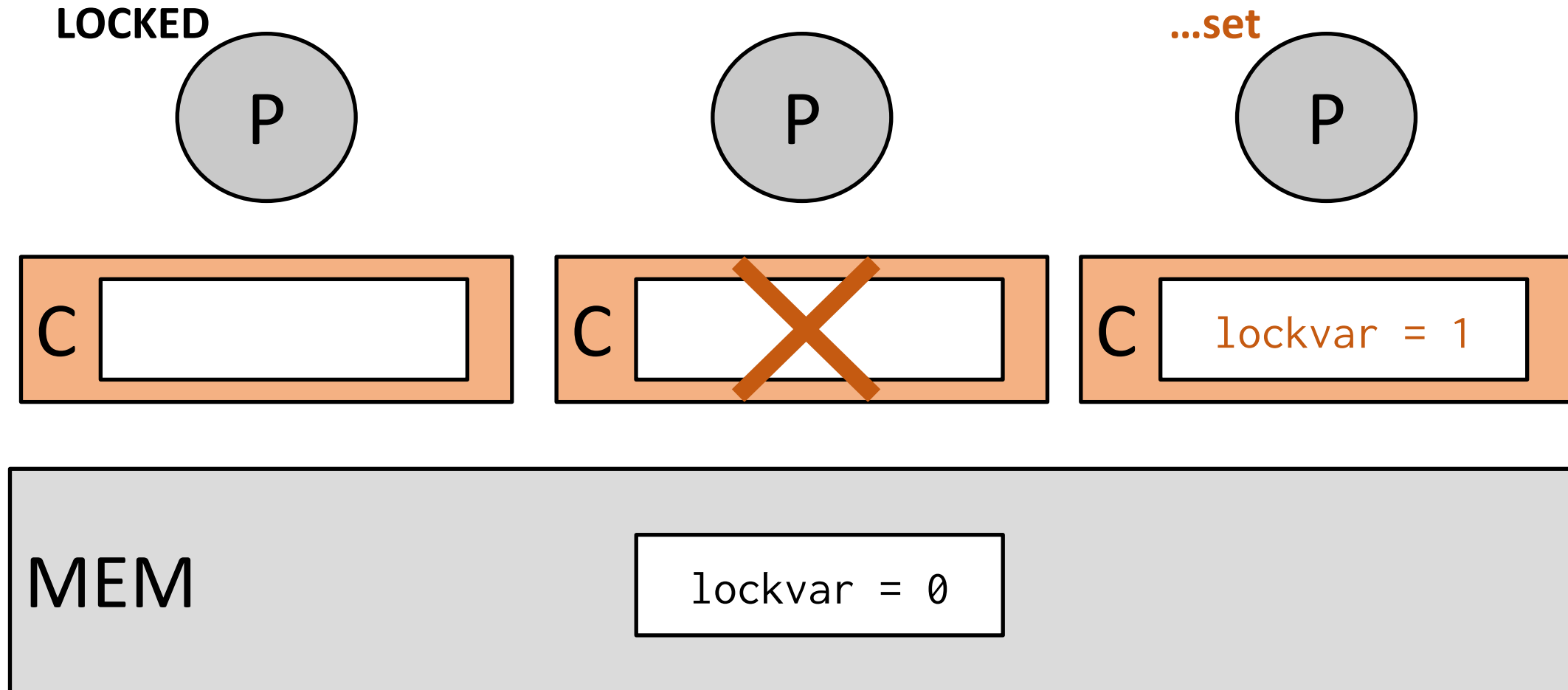
Test-And-Set Lock



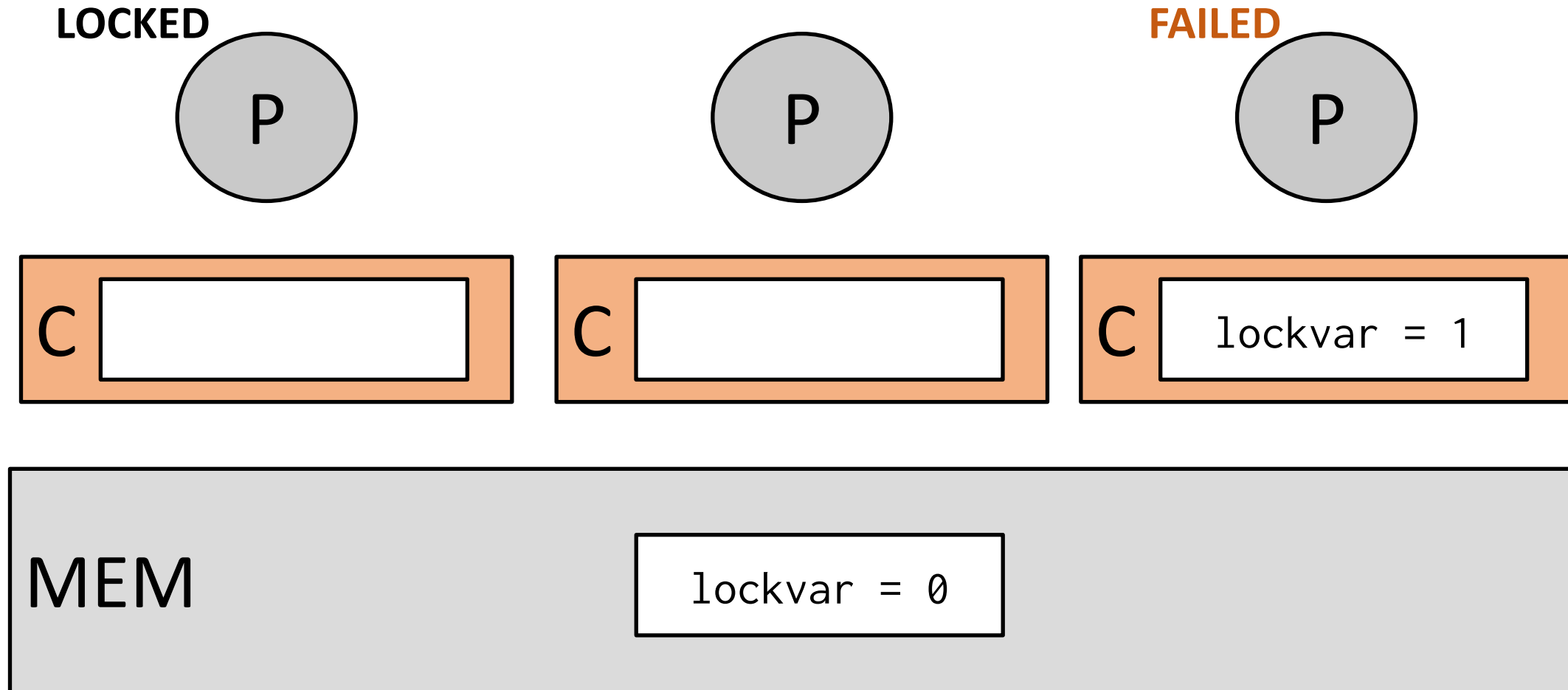
Test-And-Set Lock



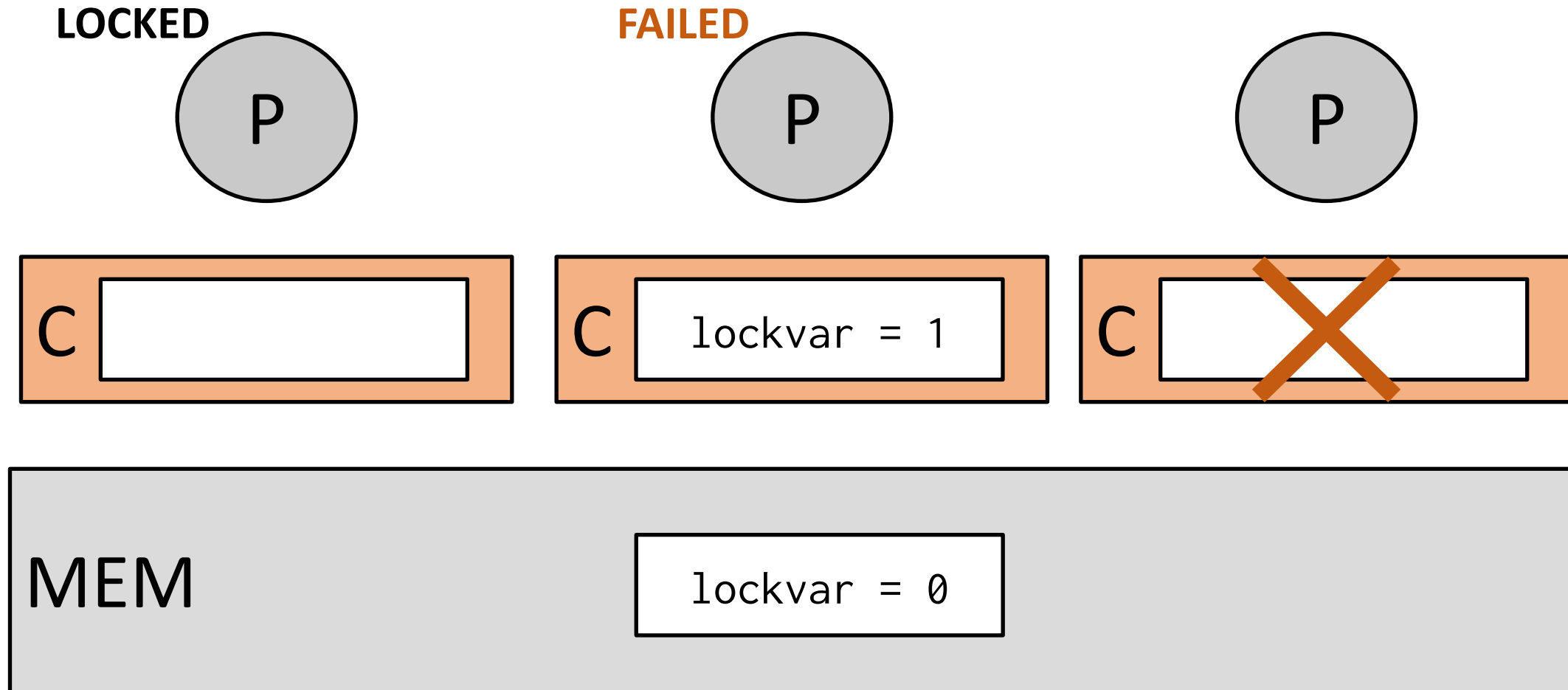
Test-And-Set Lock



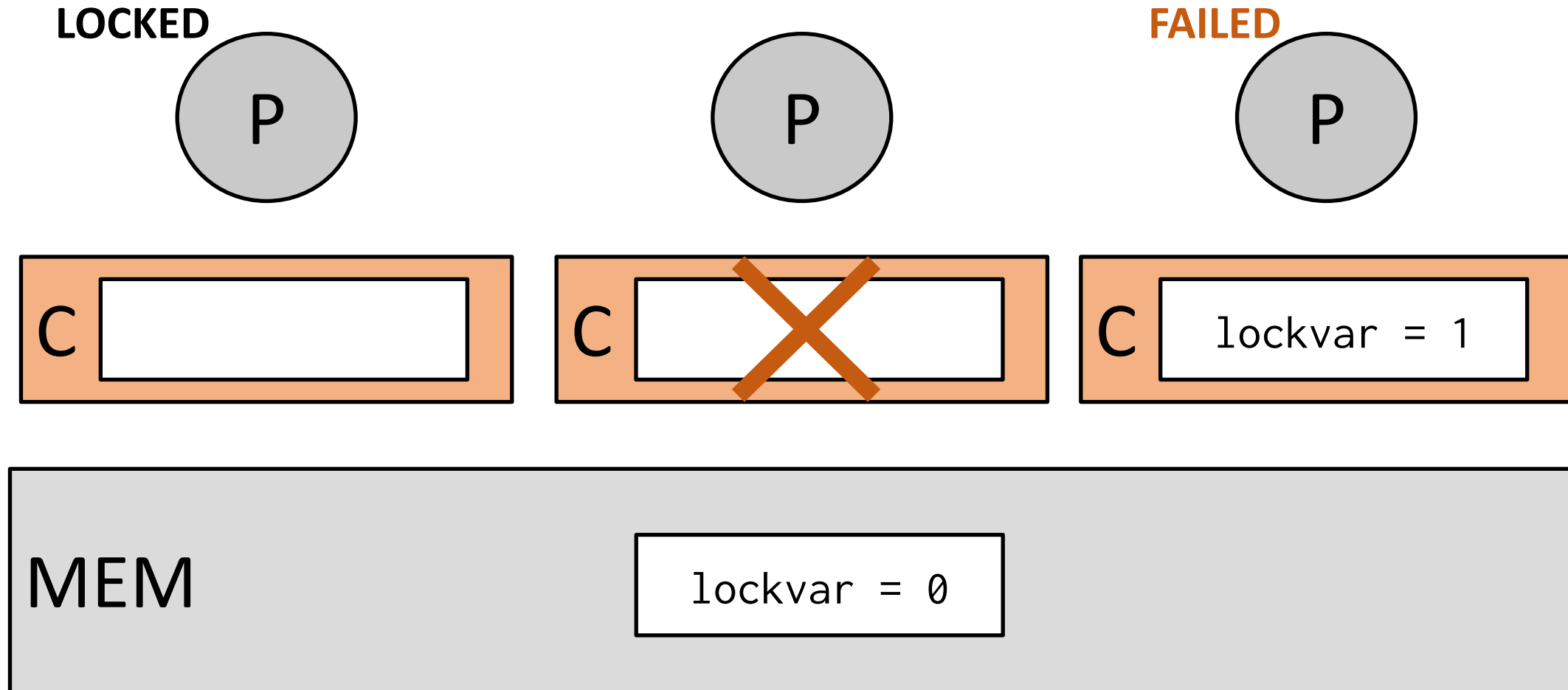
Test-And-Set Lock



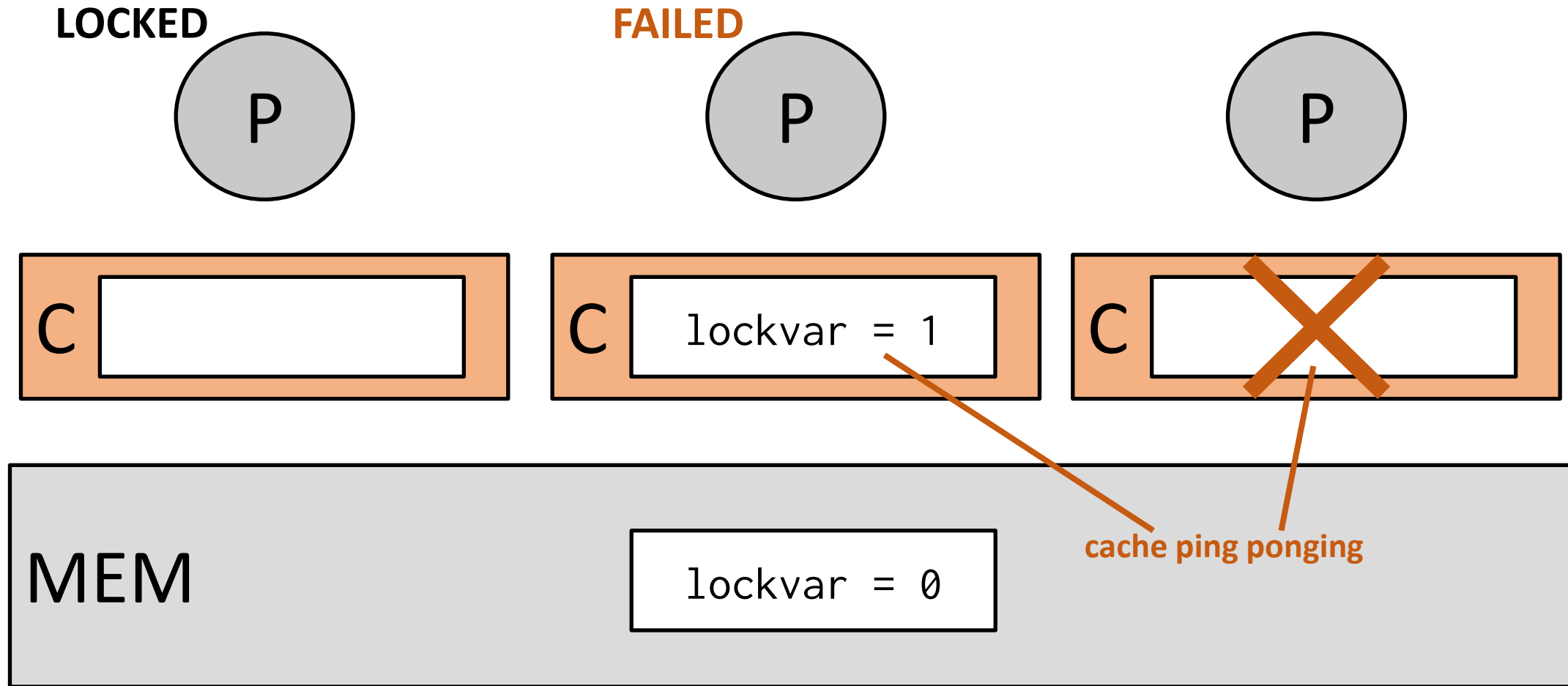
Test-And-Set Lock



Test-And-Set Lock



Test-And-Set Lock



Test-And-Test-And-Set Lock

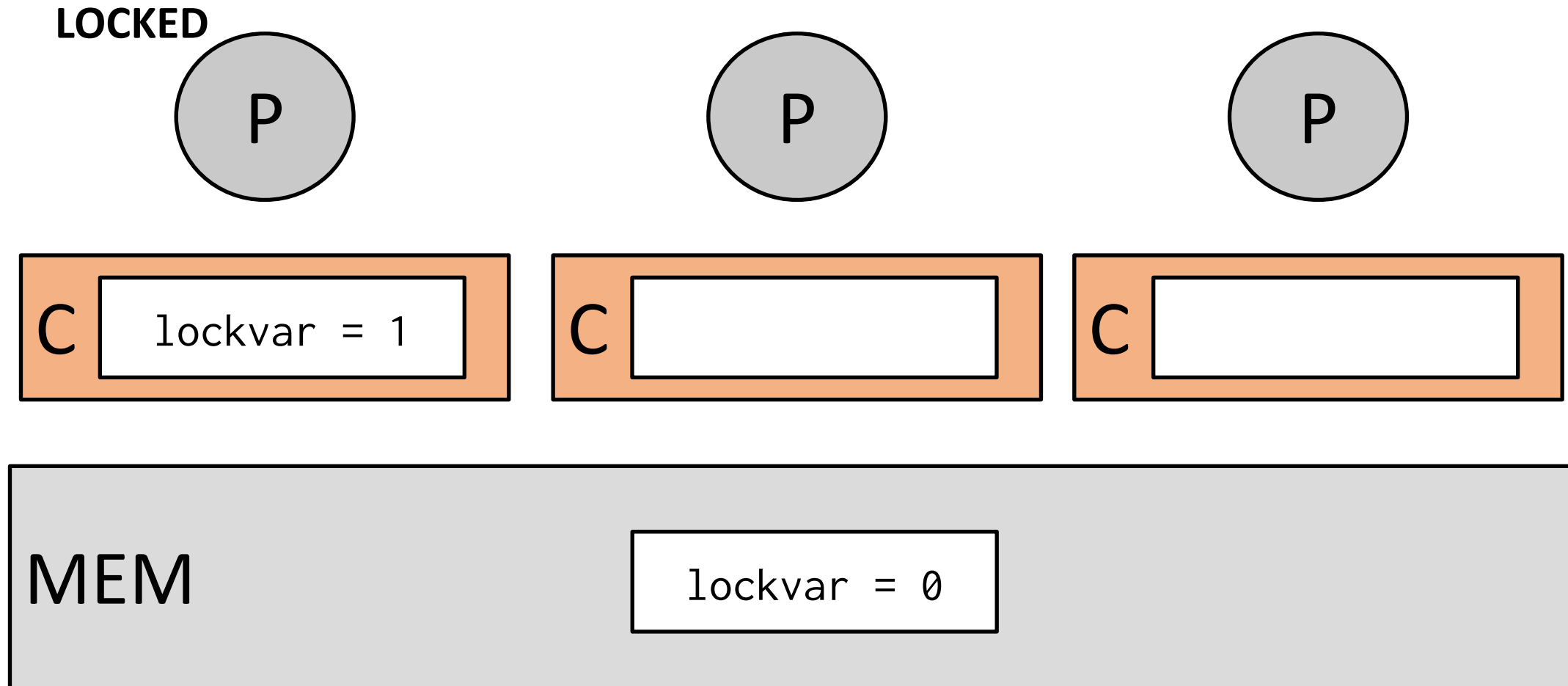
LOCK:

```
while(1) {  
    if (*lockvar == 0) { // test  
        old = test-and-set(lockvar);  
        if(old == 0) break;  
    }  
}
```

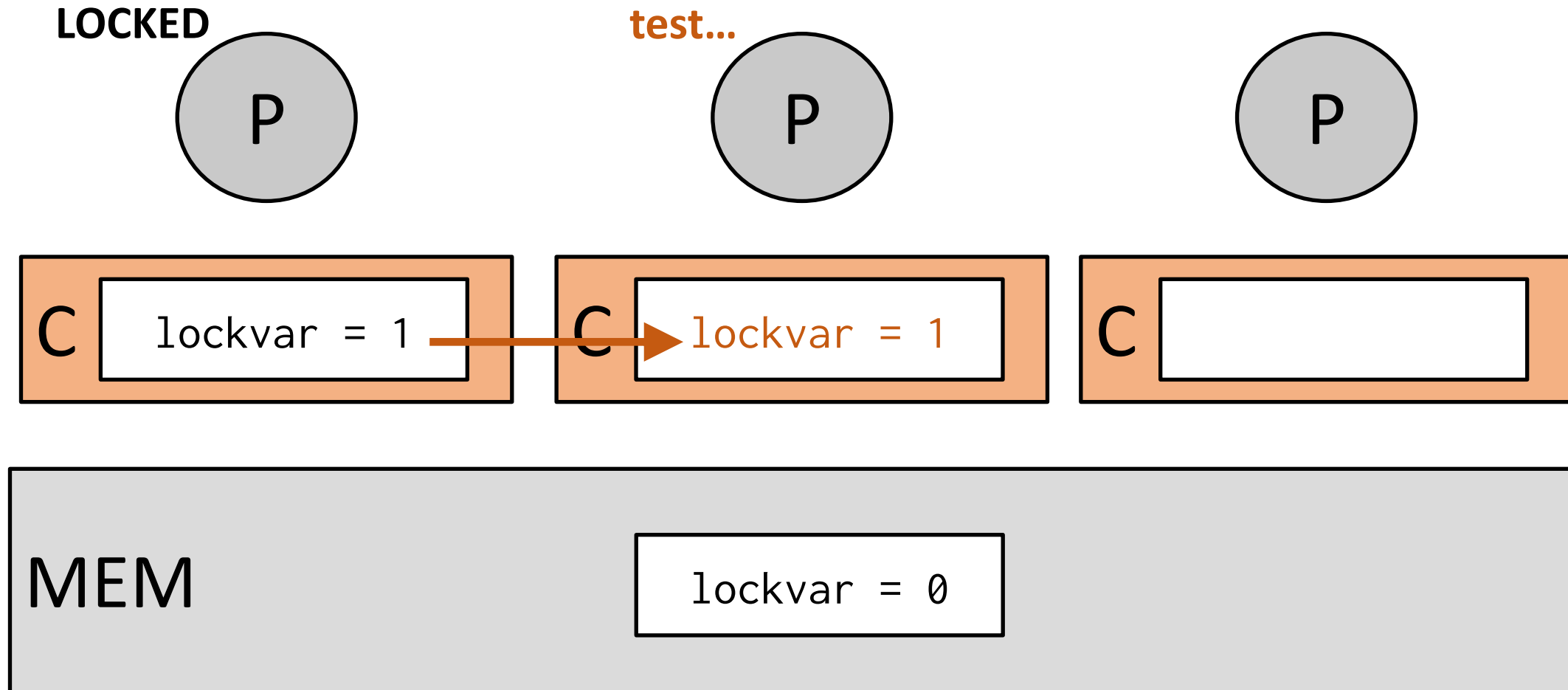
UNLOCK:

```
*lockvar = 0;
```

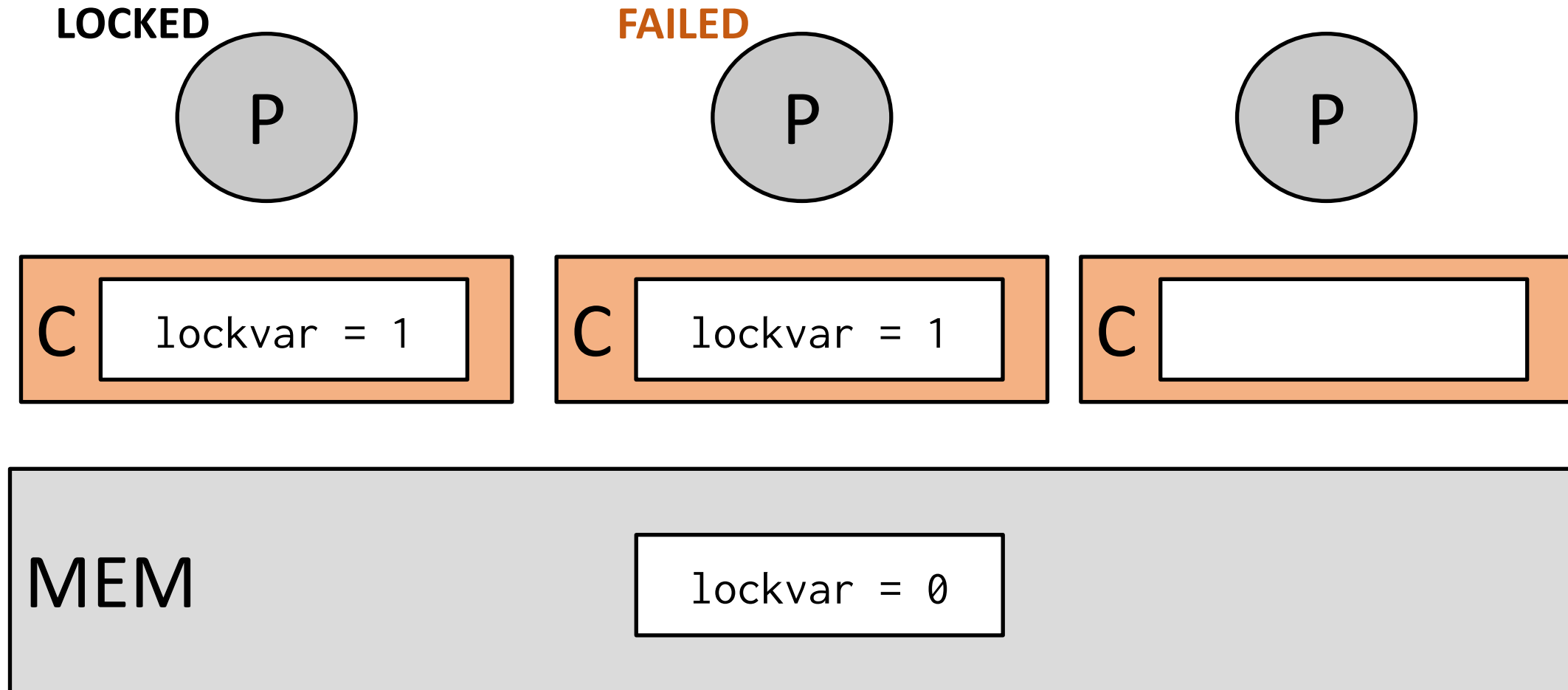
Test-And-Test-And-Set Lock



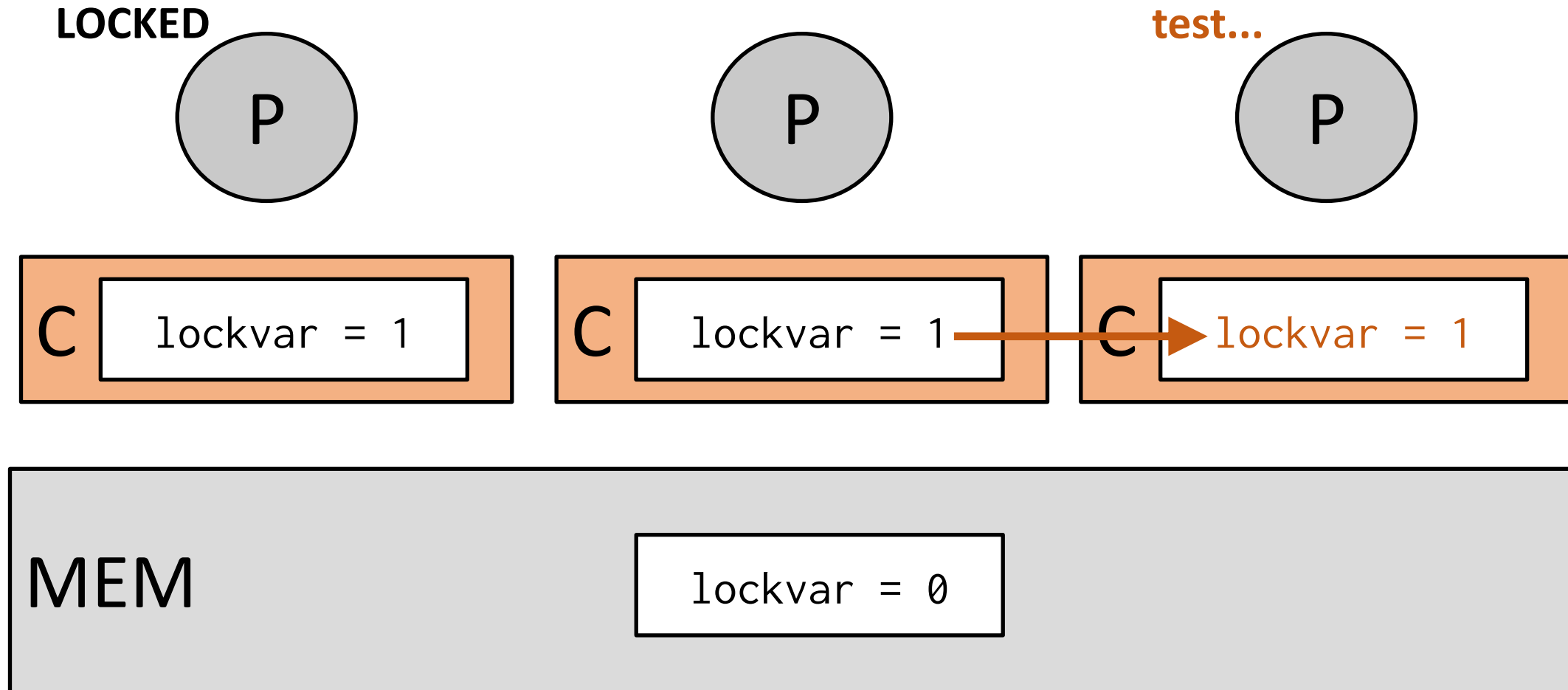
Test-And-Test-And-Set Lock



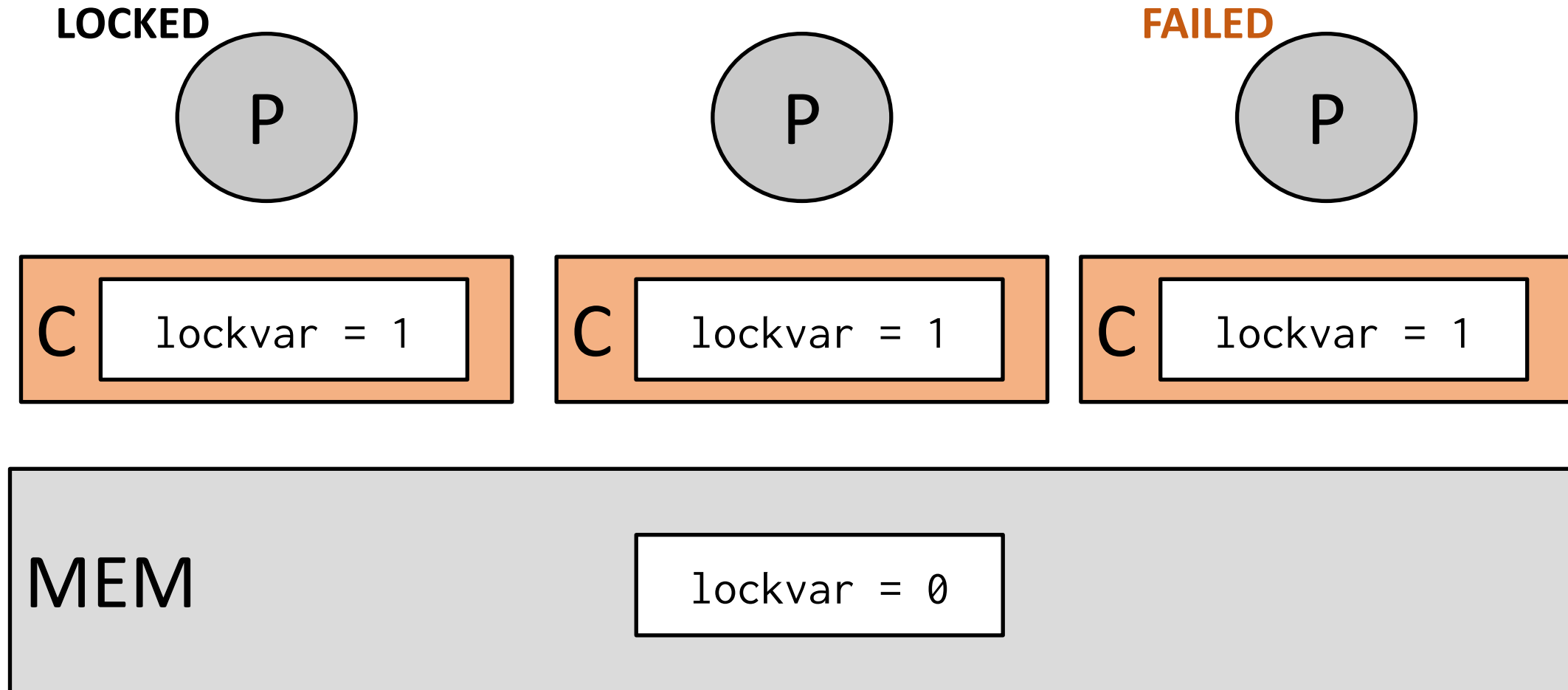
Test-And-Test-And-Set Lock



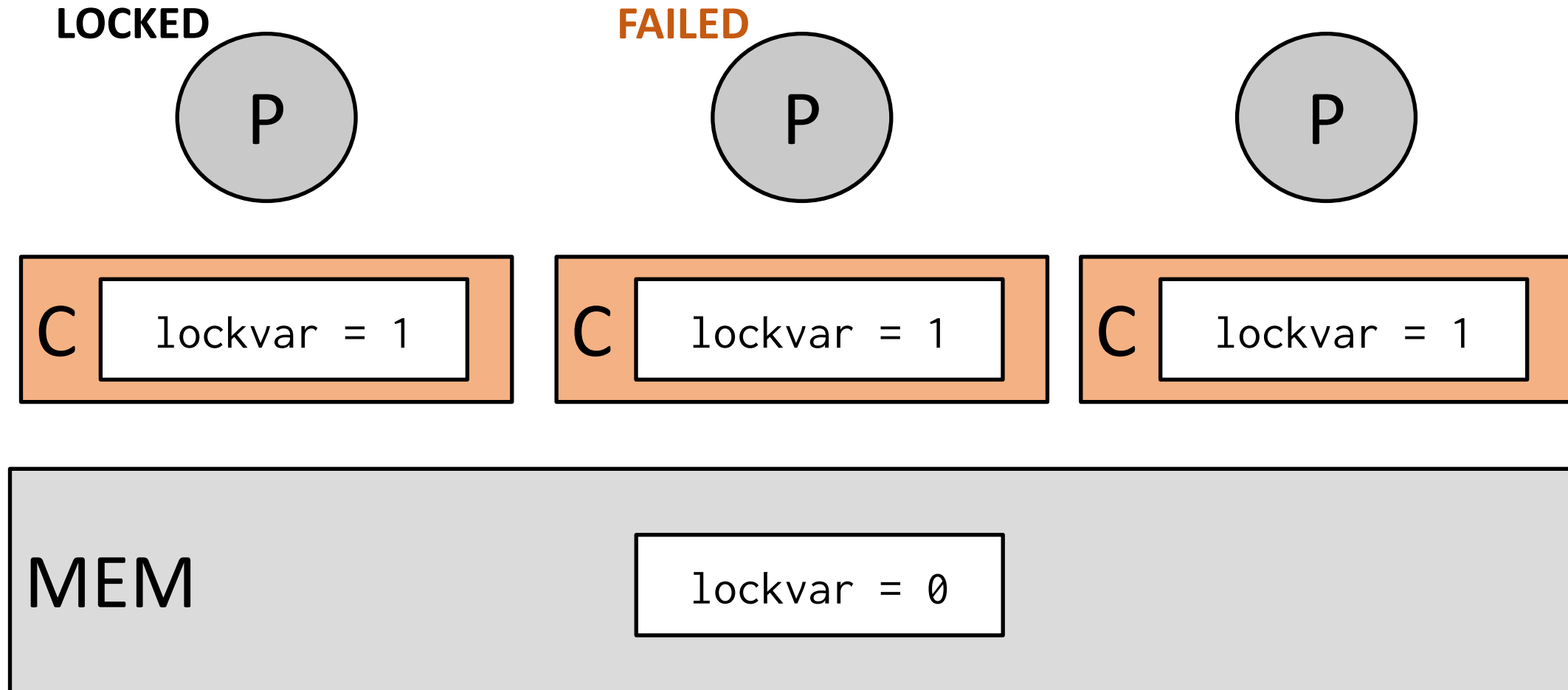
Test-And-Test-And-Set Lock



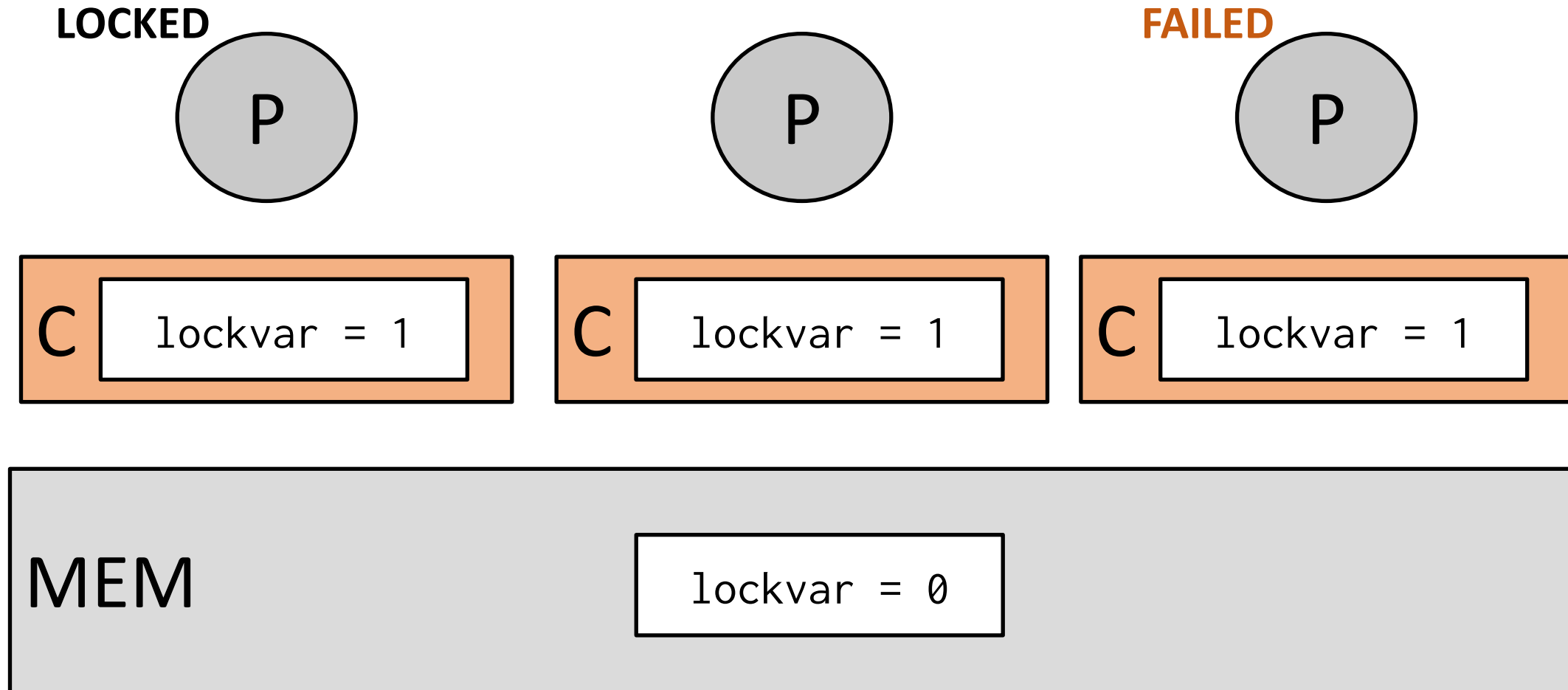
Test-And-Test-And-Set Lock



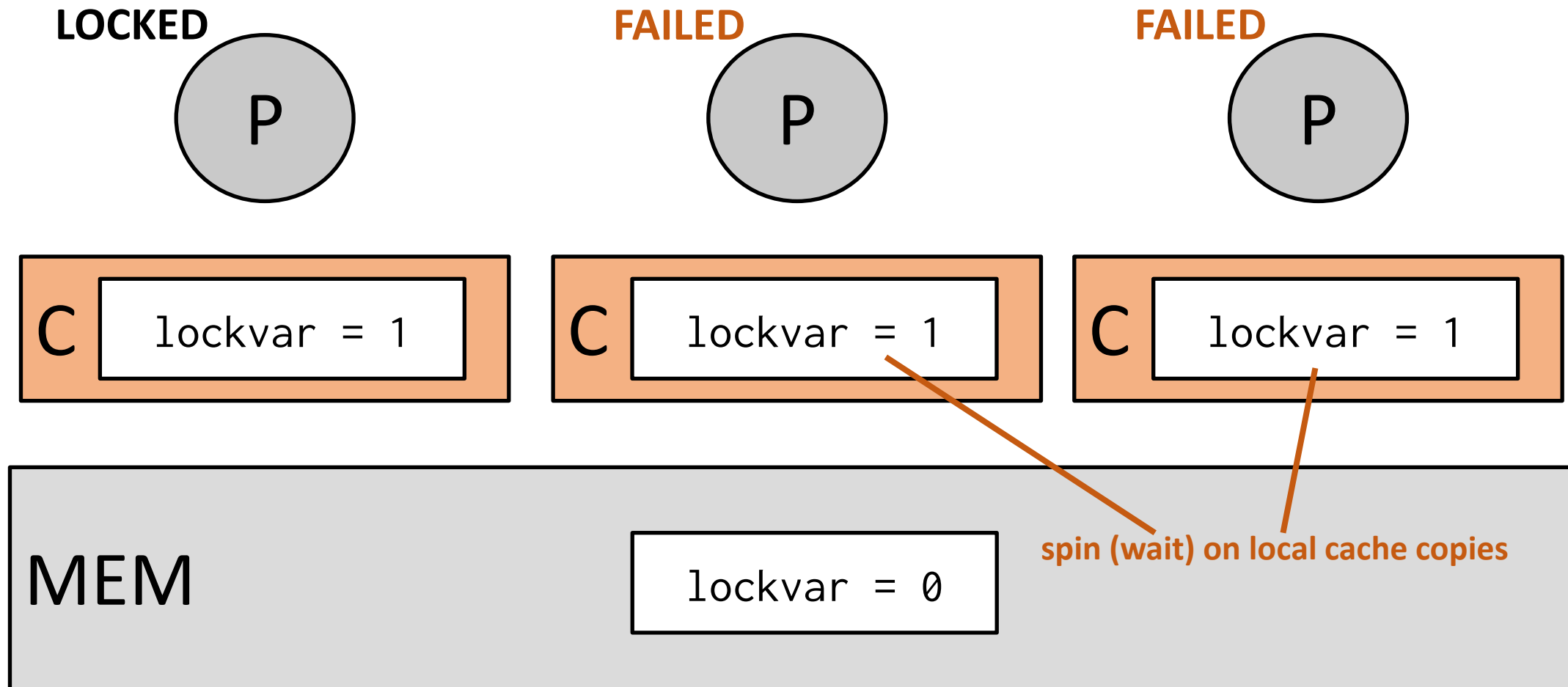
Test-And-Test-And-Set Lock



Test-And-Test-And-Set Lock



Test-And-Test-And-Set Lock



Load-Linked/Store-Conditional Instructions

Load-linked (LL):

Performs a load and sets a watchdog flag on the load address.

```
{ r1=*addr; watchflag=1; watchaddr=addr; }
```

Store-conditional (SC):

Performs a store only if the watchdog flag has not been cleared by either 1) invalidation/eviction of load address or 2) context switch.

```
{  
    if watchflag==1 and watchaddr==addr then *addr=r1 and return 1;  
    else return 0;  
}
```

Load-Linked/Store-Conditional Instructions

LOCK:

```
while(1) {  
    old = load-linked(lockvar);  
    if(old == 0)  
        if(store-conditional(lockvar, 1) == 1) break;  
}
```

UNLOCK:

```
*lockvar = 0;
```

Load-Linked/Store-Conditional Instructions

Implement an atomic fetch-and-add instruction using load-linked and store-conditional instructions.

```
r1 = fetch-and-add(addr, r2):  
    { r1=*addr; *addr=r1+r2); }
```

```
int fetch-and-add (addr, r2)  
{
```

```
}
```

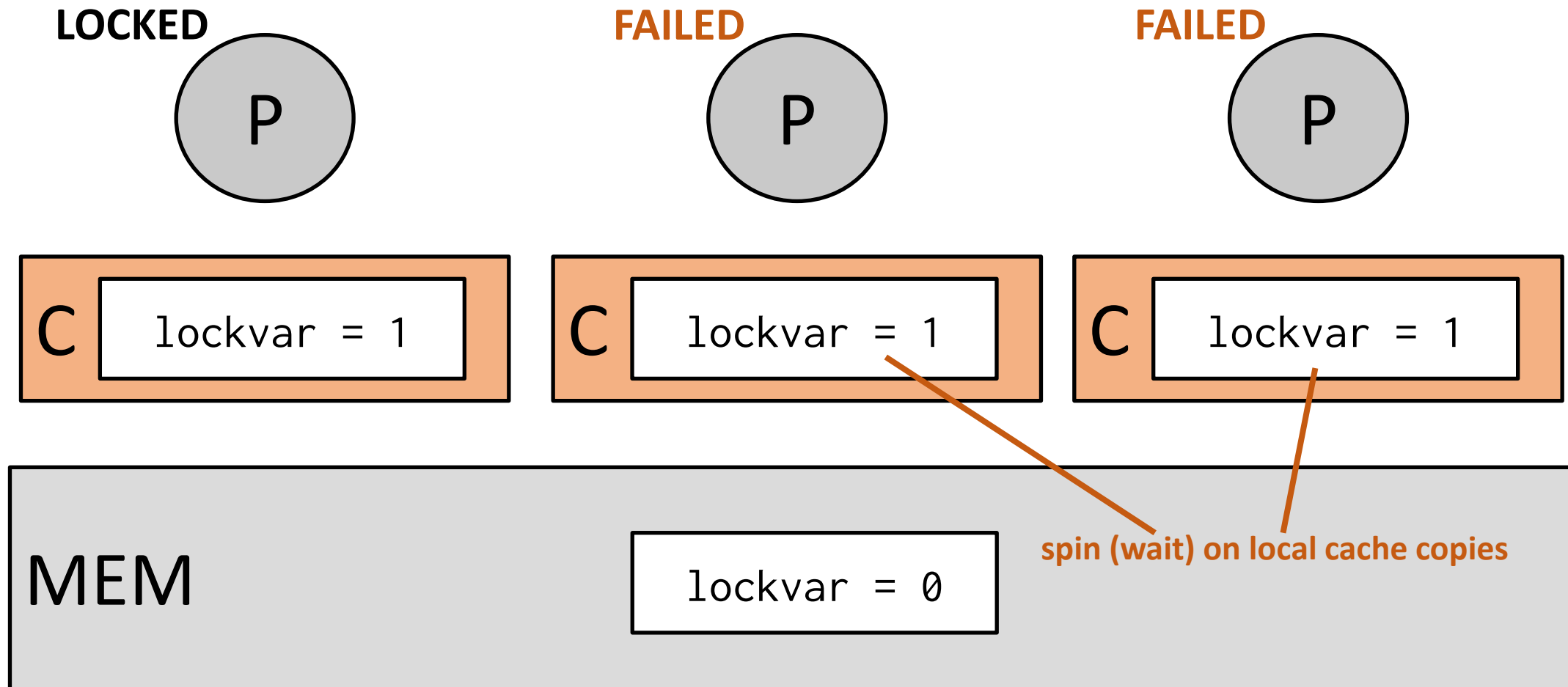
Load-Linked/Store-Conditional Instructions

Implement an atomic fetch-and-add instruction using load-linked and store-conditional instructions.

```
r1 = fetch-and-add(addr, r2):  
    { r1=*addr; *addr=r1+r2; }
```

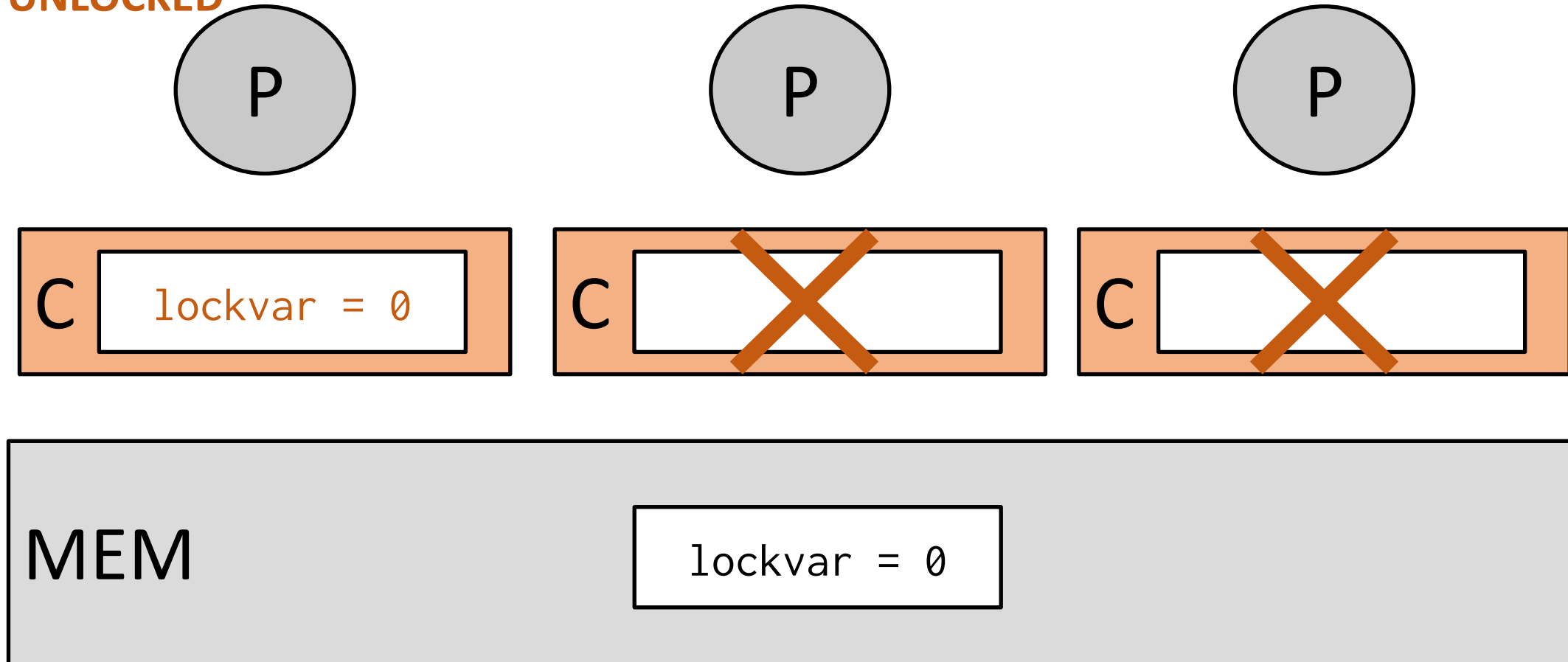
```
int fetch-and-add (addr, r2)  
{  
    while(1) {  
        r1 = load-linked(addr);  
        tmp = r1 + r2;  
        if(store-conditional(addr, tmp) == 1) break;  
    }  
    return r1;  
}
```

Test-And-Test-And-Set Lock

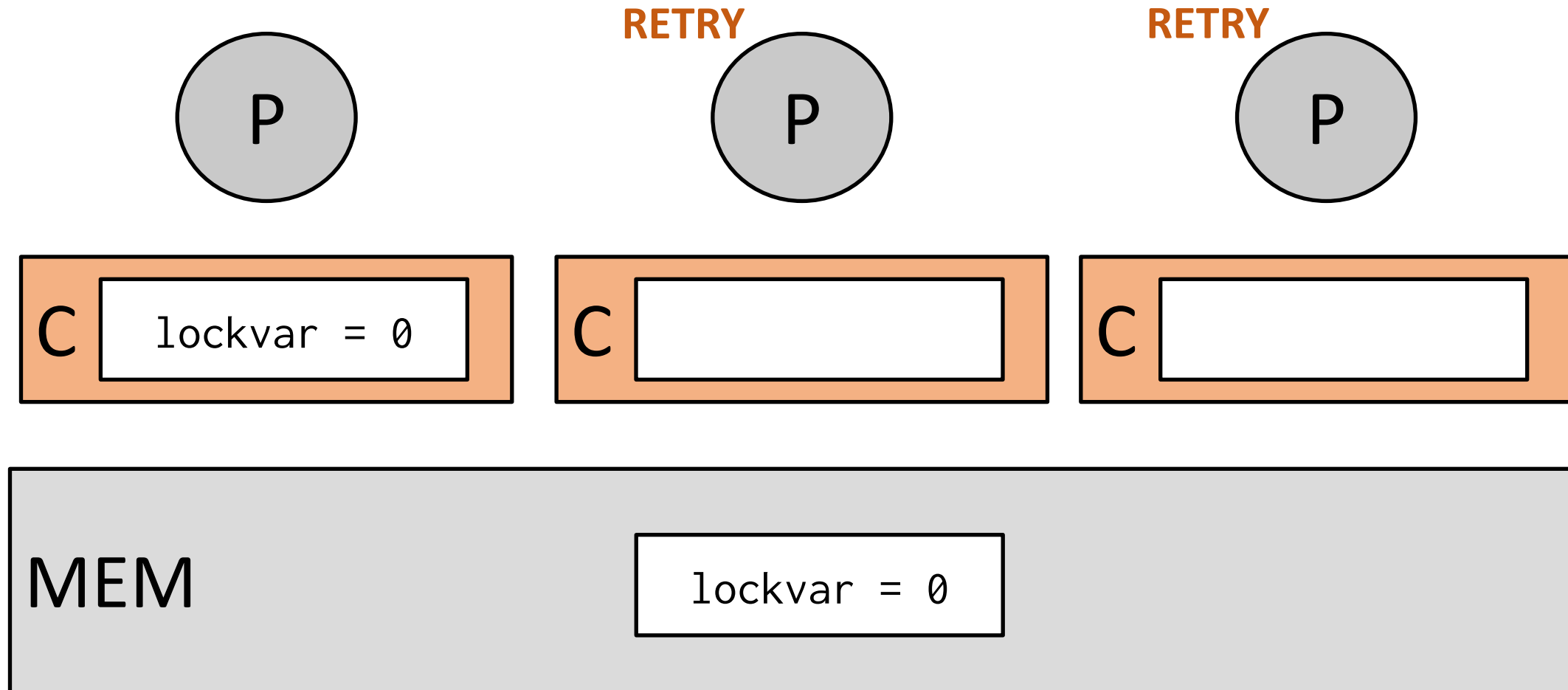


Test-And-Test-And-Set Lock

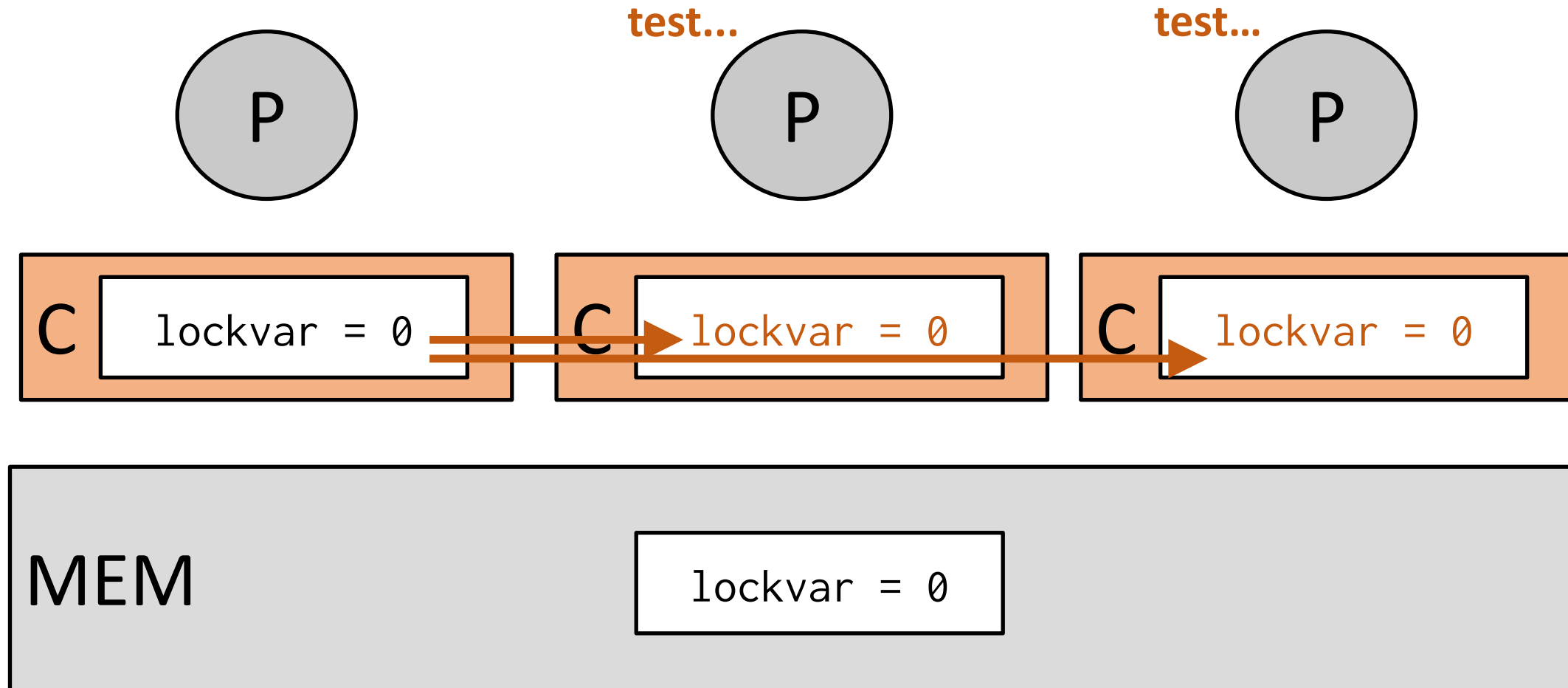
UNLOCKED



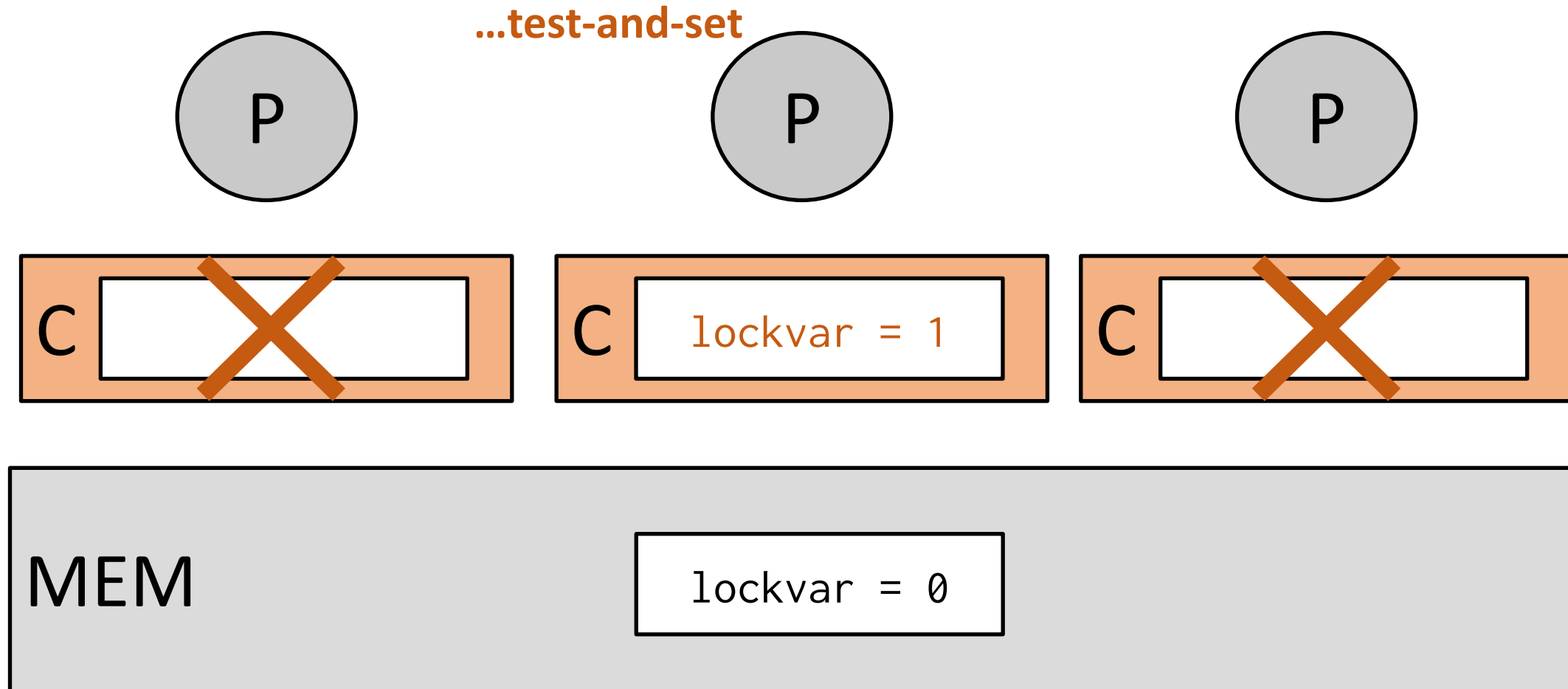
Test-And-Test-And-Set Lock



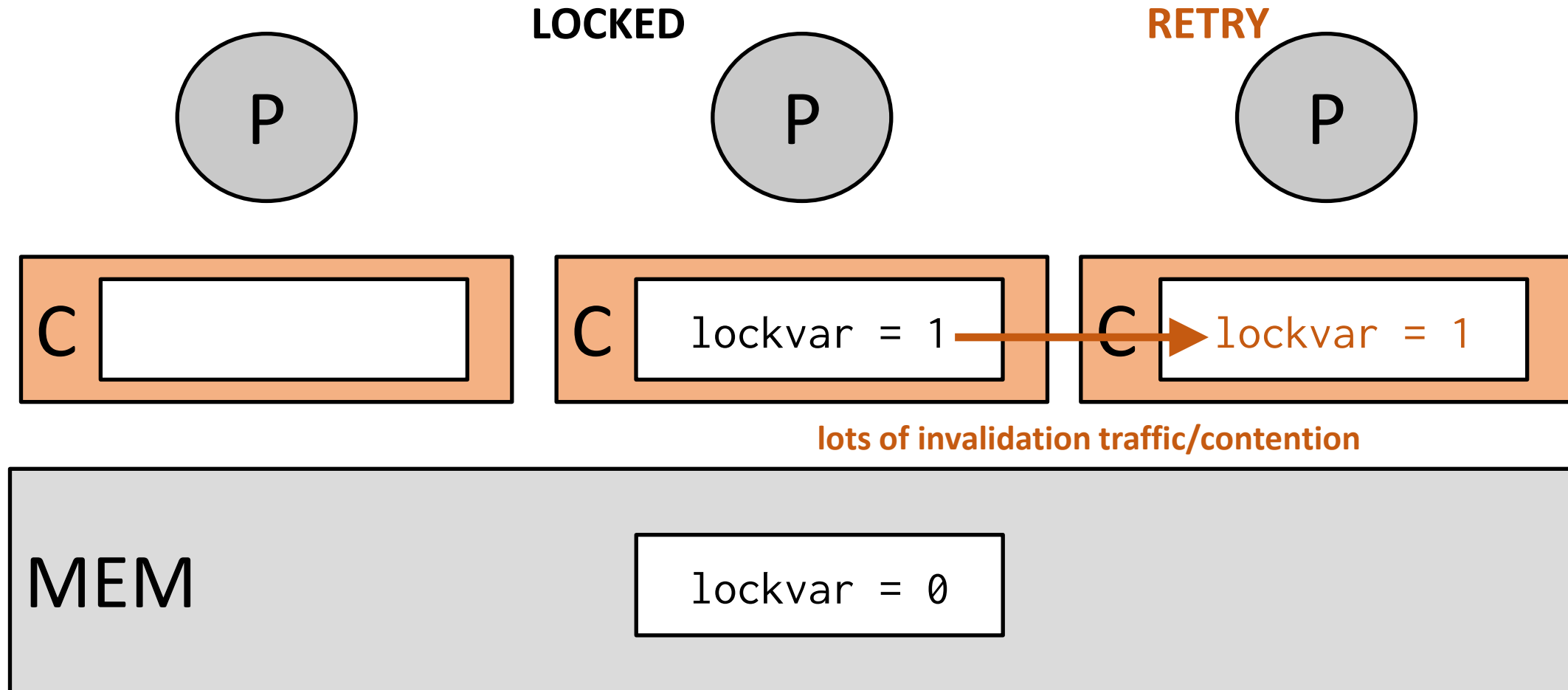
Test-And-Test-And-Set Lock



Test-And-Test-And-Set Lock



Test-And-Test-And-Set Lock



Test-And-Test-And-Set Lock with Backoff

LOCK:

```
    delay = 1;
    while(1) {
        if(*lockvar == 0) { // test
            old = test-and-set(lockvar);
            if(old == 0) break;
        }
        pause(delay); // or sleep
        delay = delay * 2;
    }
```

UNLOCK:

```
*lockvar = 0;
```

Test-And-Test-And-Set Lock with Backoff

LOCK:

```
    delay = 1;
    while(1) {
        if(*lockvar == 0) { // test
            old = test-and-set(lockvar);
            if(old == 0) break;
        }
        pause(delay); // or sleep
        delay = delay * 2;
    }
```

Is this fair?

UNLOCK:

```
*lockvar = 0;
```

Ticket Lock

LOCK:

```
myticket = fetch-and-add(&lockvar->nextticket, 1);  
while(1) {  
    if(lockvar->nowserving == myticket) break;  
}
```

UNLOCK:

```
lockvar->nowserving = lockvar->nowserving + 1;
```

Ticket Lock

LOCK:

```
myticket = fetch-and-add(&lockvar->nextticket, 1);  
while(1) {  
    if(lockvar->nowserving == myticket) break;  
}
```

UNLOCK:

```
lockvar->nowserving = lockvar->nowserving + 1;
```

is this correct?

Ticket Lock

LOCK:

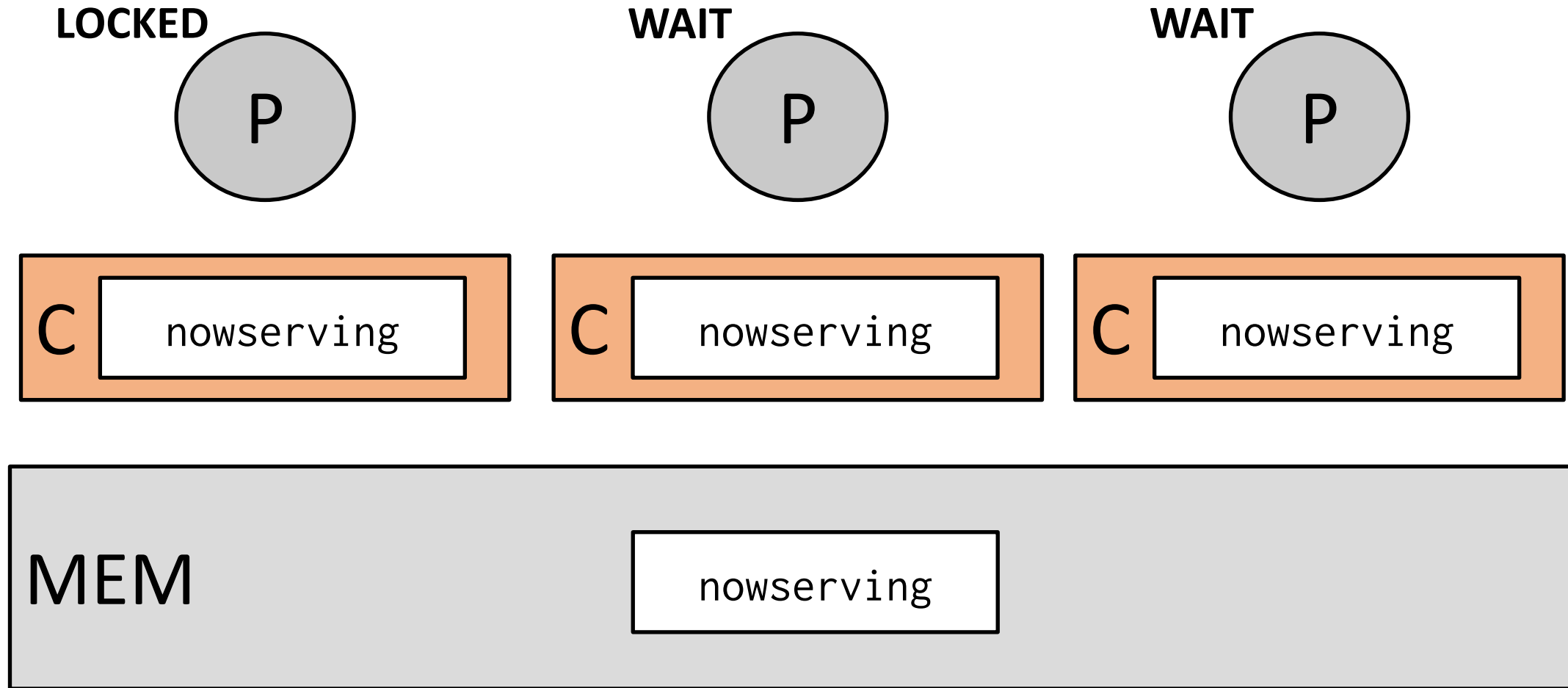
```
myticket = fetch-and-add(&lockvar->nextticket, 1);  
while(1) {  
    if(lockvar->nowserving == myticket) break;  
}
```

UNLOCK:

```
lockvar->nowserving = lockvar->nowserving + 1;
```

Enforces first-in-first-out order

Ticket Lock



Ticket Lock

LOCK:

can we use fetch-and-store instead?

```
myticket = fetch-and-add(&lockvar->nextticket, 1);  
while(1) {  
    if(lockvar->nowserving == myticket) break;  
}
```

UNLOCK:

```
lockvar->nowserving = lockvar->nowserving + 1;
```

Ticket Lock

LOCK:

```
while(1) {  
    tmp = lockvar->nextticket;  
    myticket = fetch-and-store(&lockvar->nextticket, tmp+1);  
    if (myticket == tmp) break;  
}  
while(1) {  
    if(lockvar->nowserving == myticket) break;  
}
```

UNLOCK:

```
lockvar->nowserving = lockvar->nowserving + 1;
```



Ticket Lock

LOCK:

does this actually work?

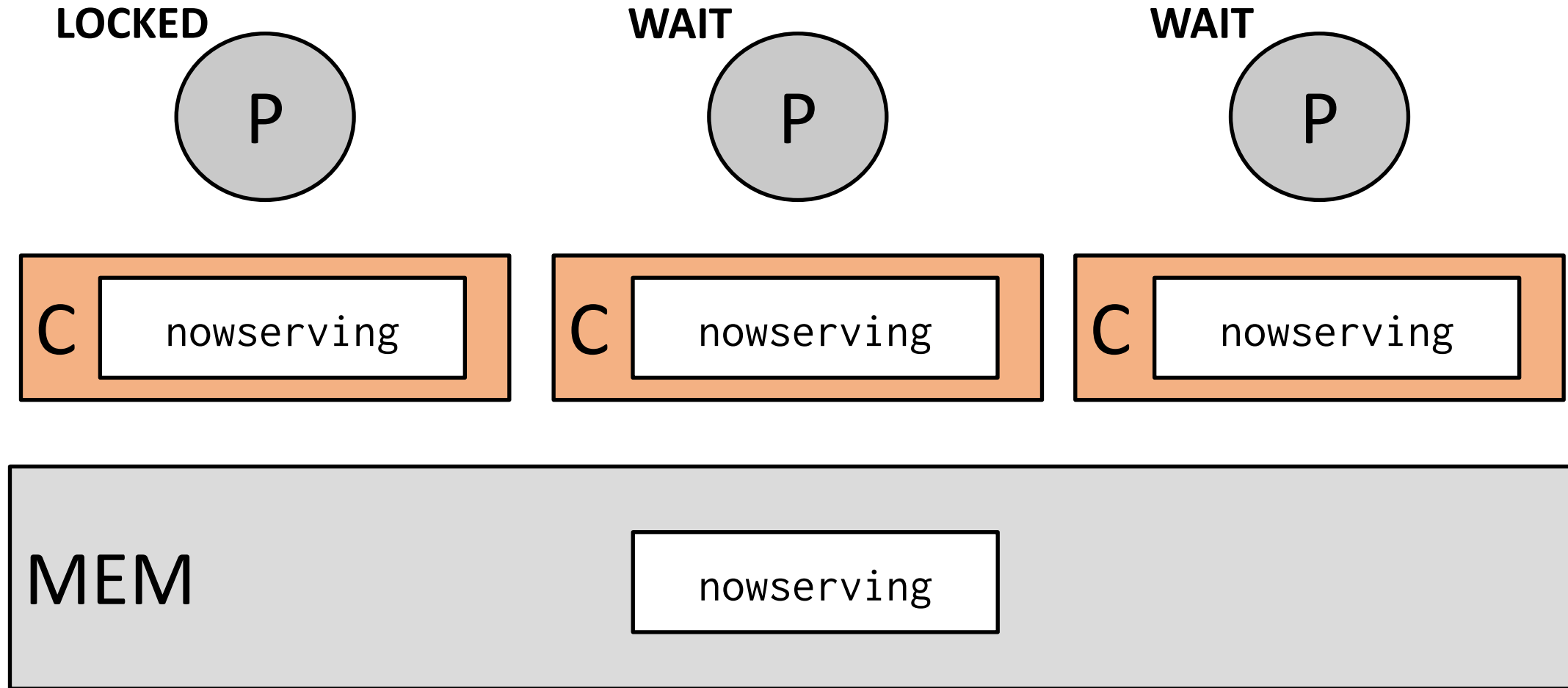
```
while(1) {  
    tmp = lockvar->nextticket;  
    myticket = fetch-and-store(&lockvar->nextticket, tmp+1);  
    if (myticket == tmp) break;  
}  
while(1) {  
    if(lockvar->nowserving == myticket) break;  
}
```

UNLOCK:

```
lockvar->nowserving = lockvar->nowserving + 1;
```

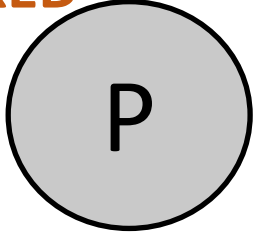


Ticket Lock

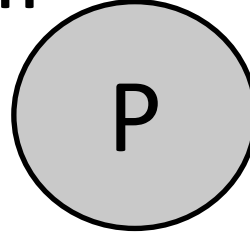


Ticket Lock

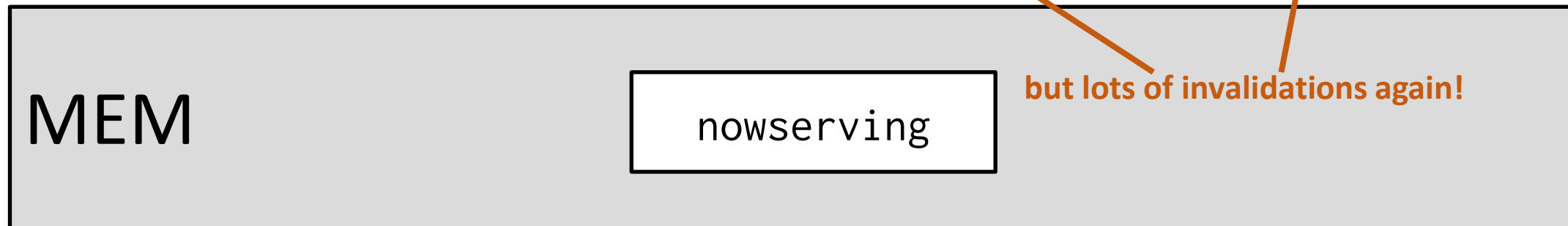
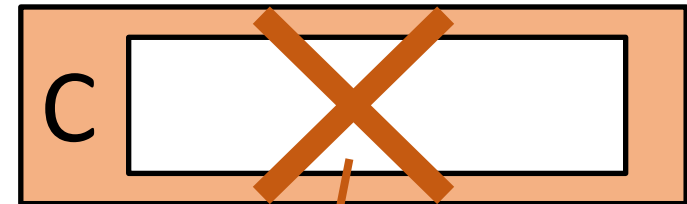
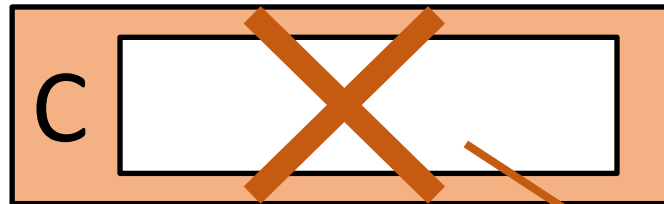
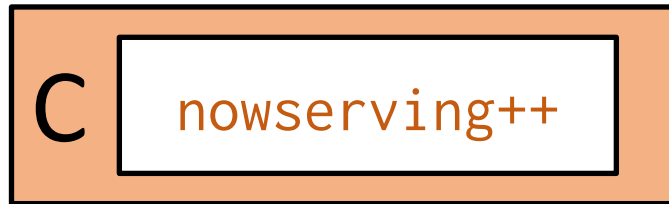
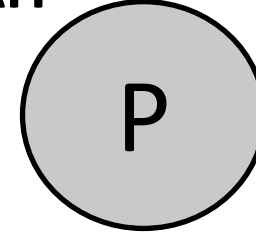
UNLOCKED



WAIT



WAIT



but lots of invalidations again!



Ticket Lock

LOCK:

```
myticket = fetch-and-add(&lockvar->nextticket, 1);  
while(1) {  
    if(lockvar->nowserving == myticket) break;  
}
```

UNLOCK:

```
lockvar->nowserving = lockvar->nowserving + 1;
```

Would adding backoff help?

Array-Based Queuing Lock

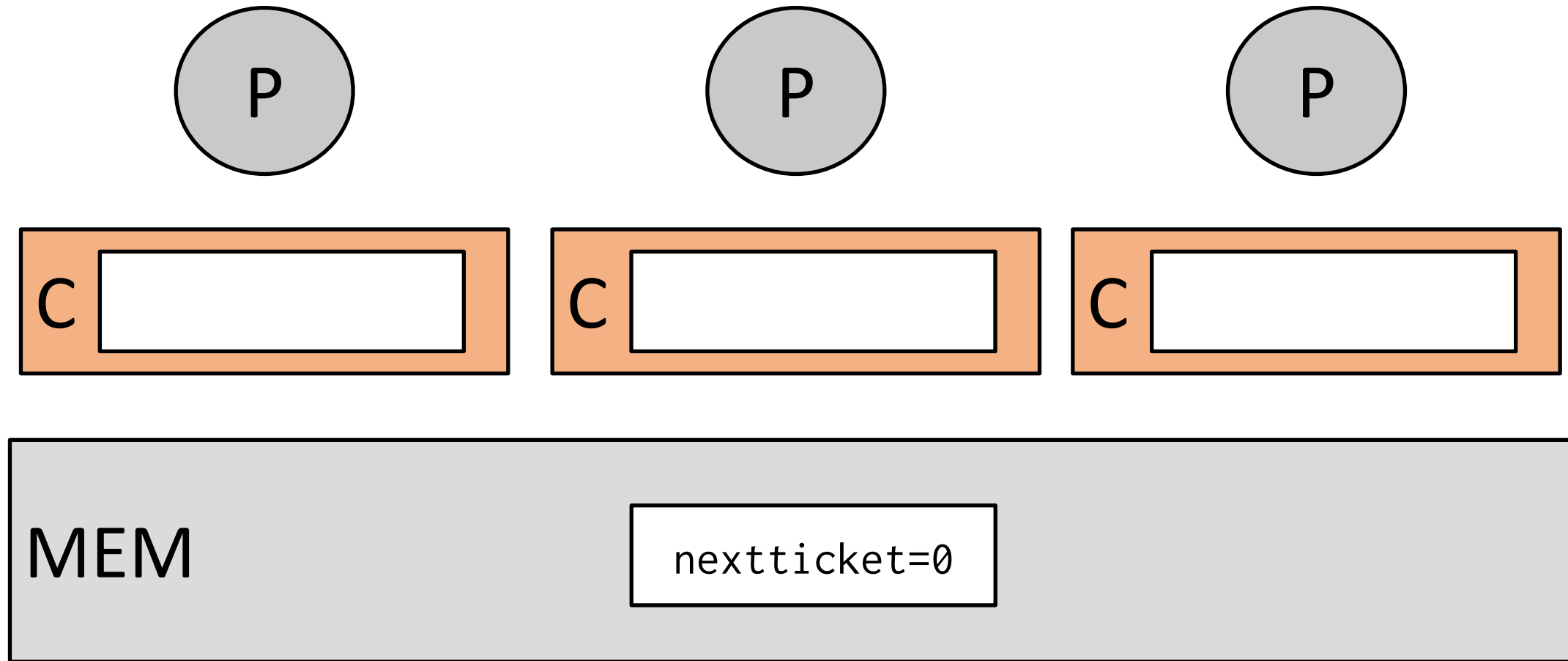
LOCK:

```
myticket = fetch-and-add(&lockvar->nextticket, 1);  
while(1) {  
    if(lockvar->isready[myticket] == 1) break;  
}
```

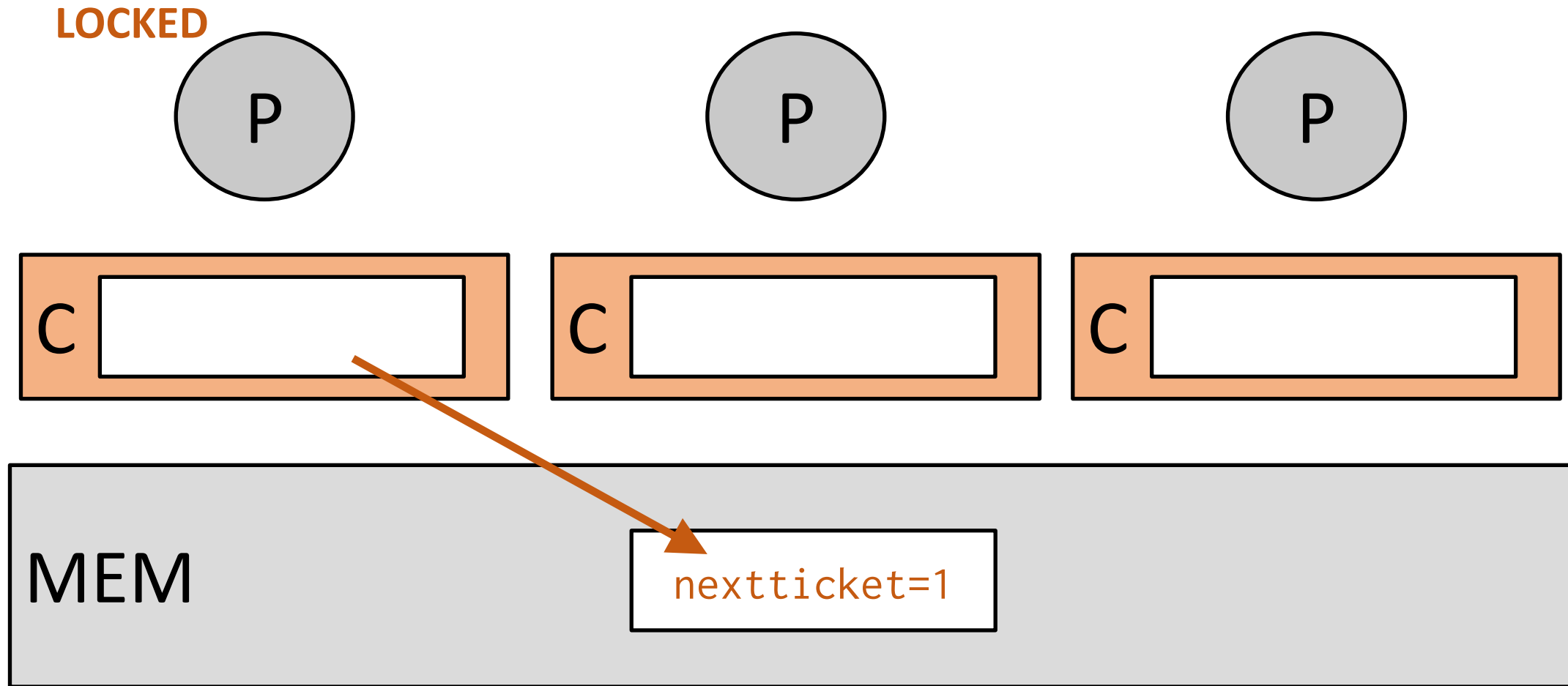
UNLOCK:

```
lockvar->isready[myticket + 1] = 1;
```

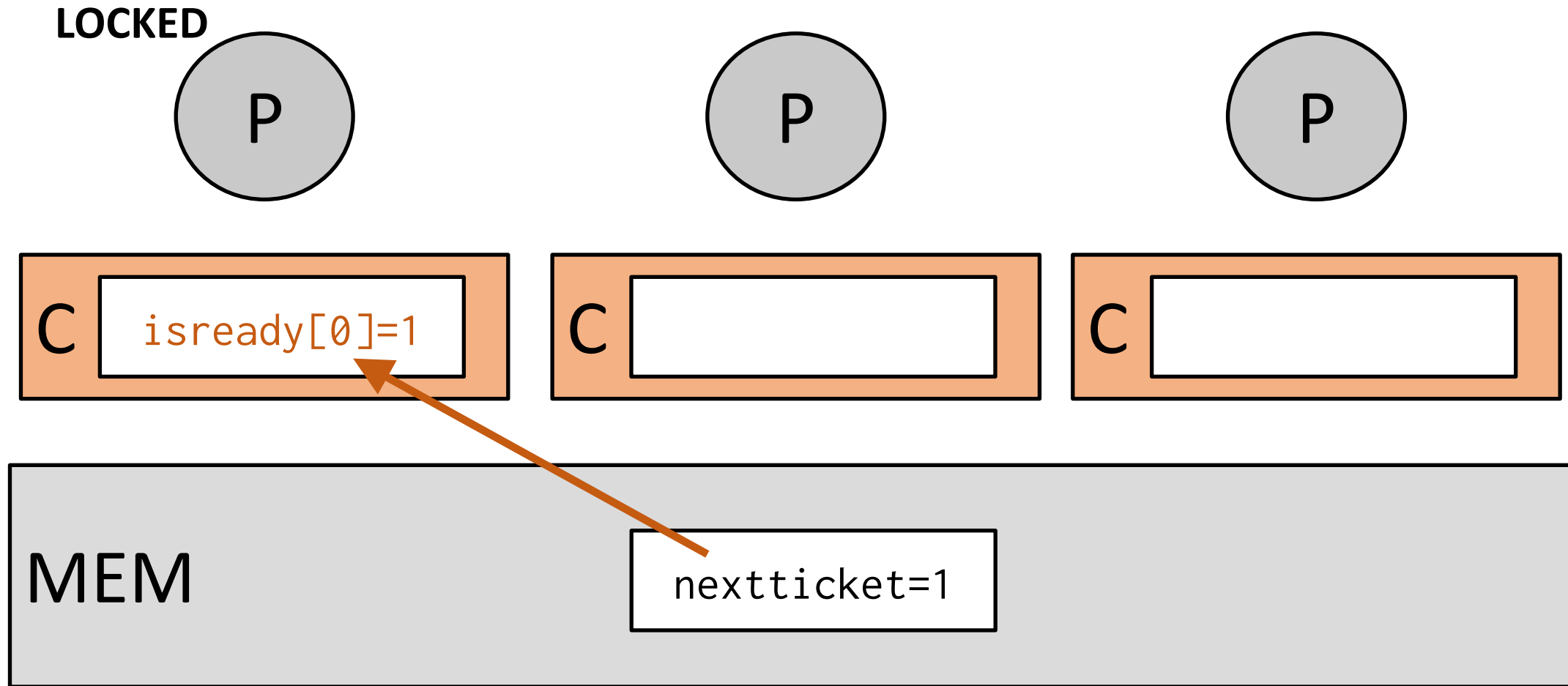
Array-Based Queuing Lock



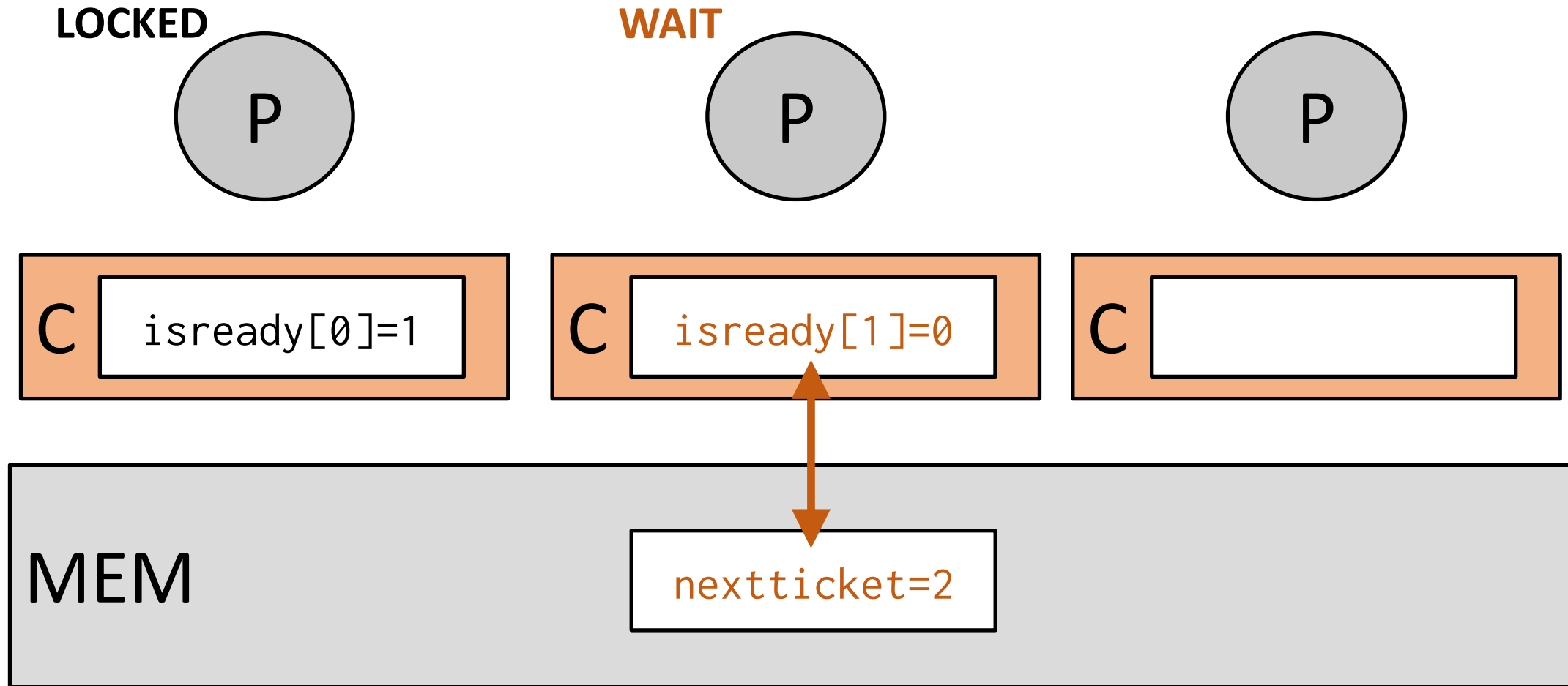
Array-Based Queuing Lock



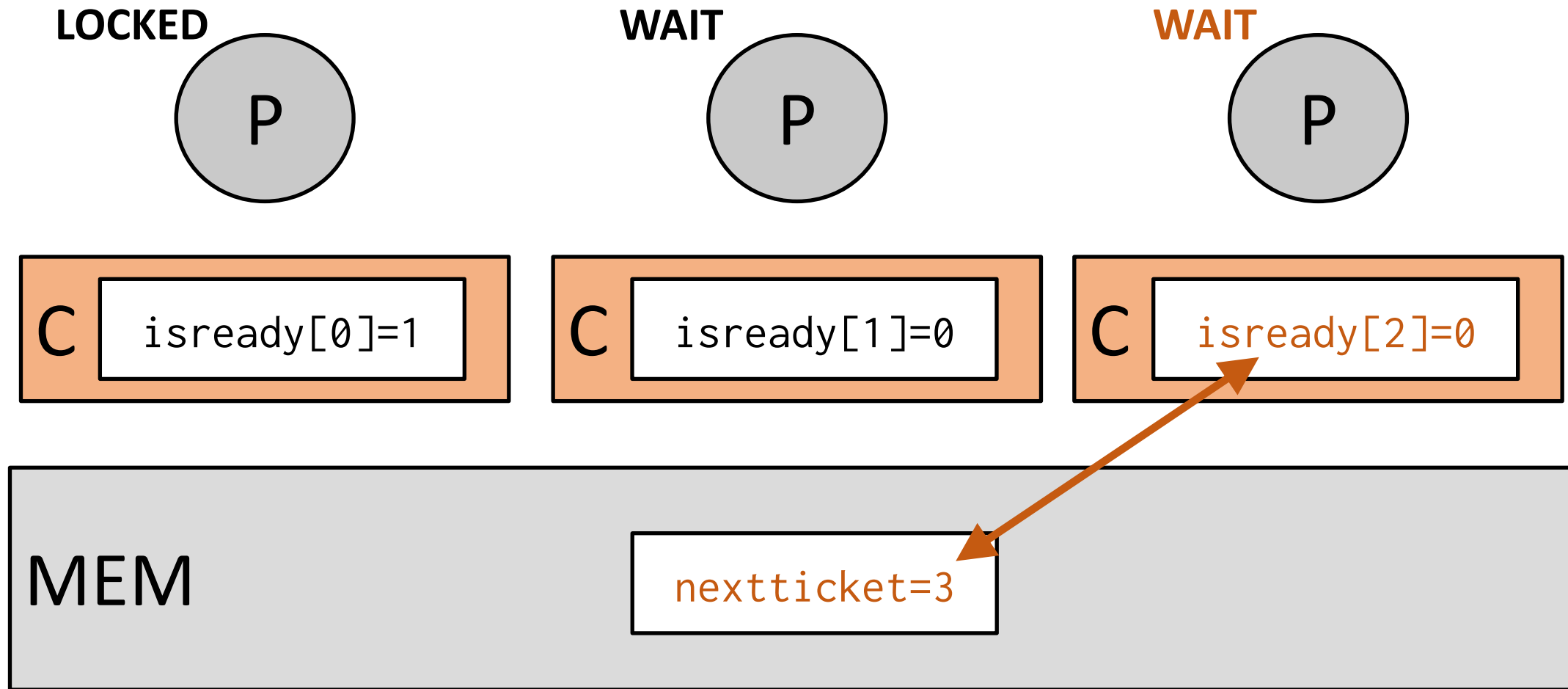
Array-Based Queuing Lock



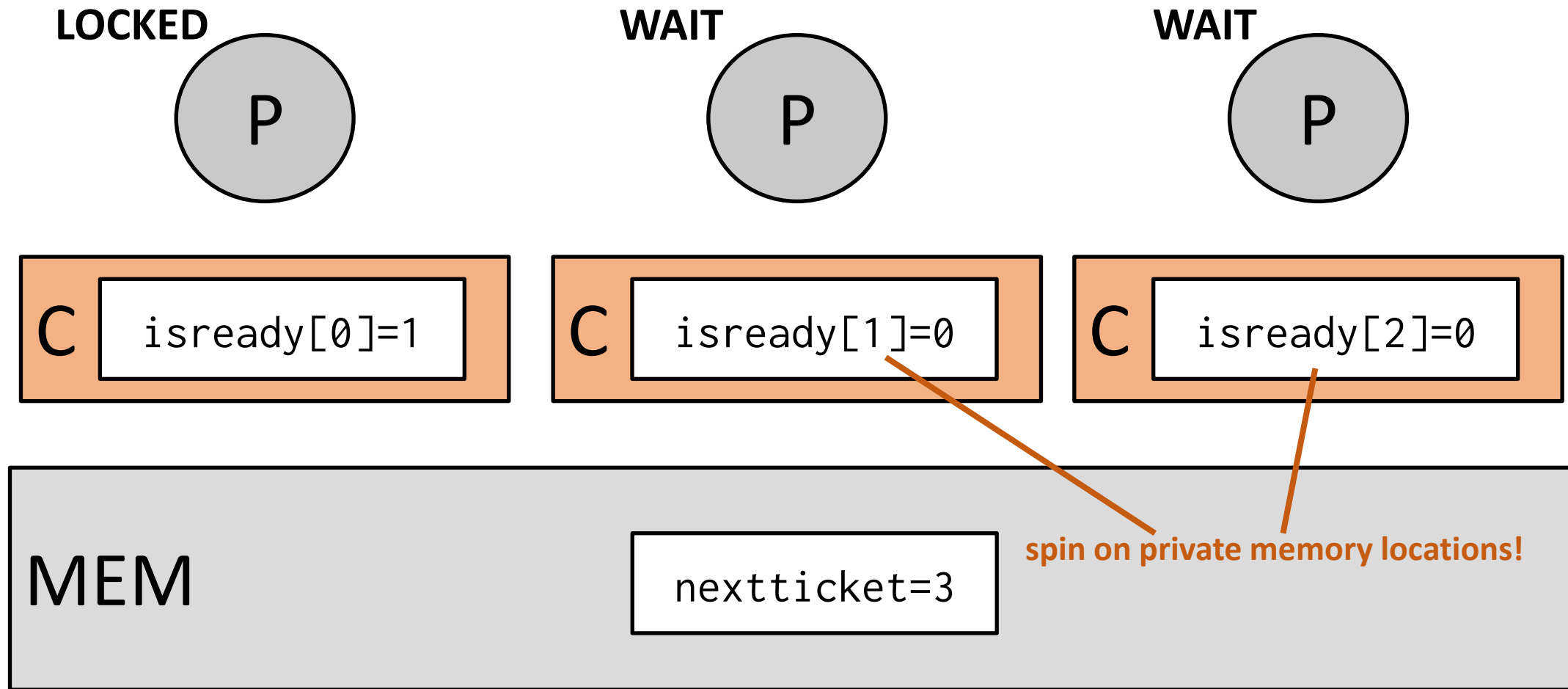
Array-Based Queuing Lock



Array-Based Queuing Lock

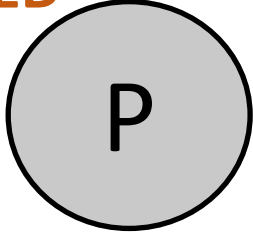


Array-Based Queuing Lock

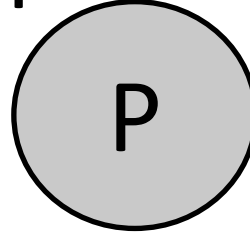


Array-Based Queuing Lock

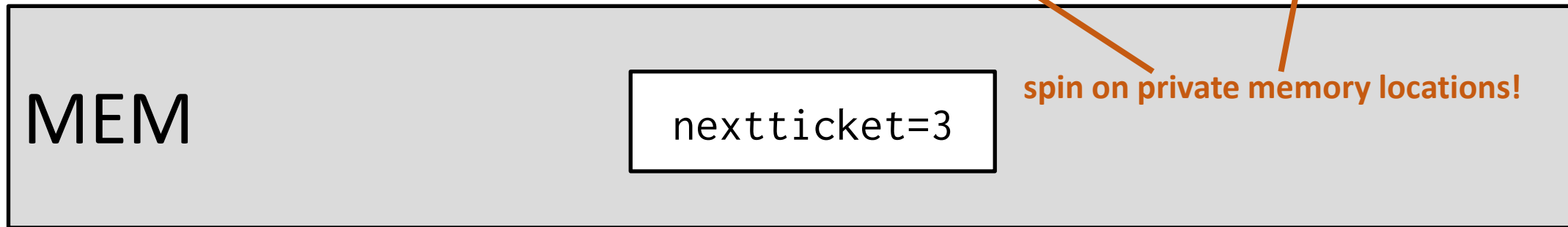
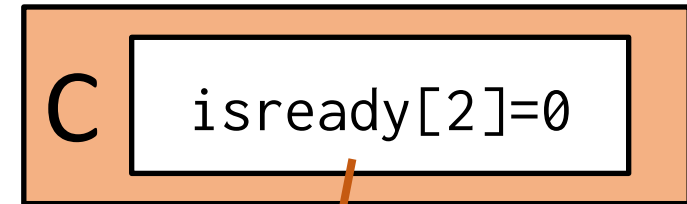
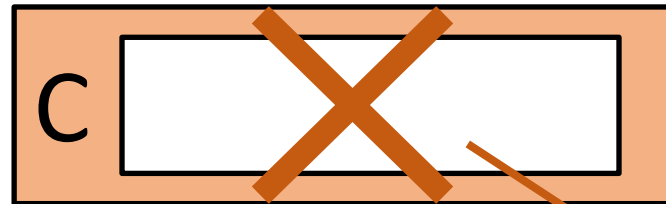
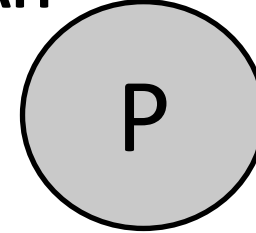
UNLOCKED



WAIT



WAIT



spin on private memory locations!



Array-Based Queuing Lock

LOCK:

```
myticket = fetch-and-add(&lockvar->nextticket, 1);  
while(1) {  
    if(lockvar->isready[myticket] == 1) break;  
}
```

UNLOCK:

```
lockvar->isready[myticket + 1] = 1;
```

Array-Based Queuing Lock

LOCK:

```
myticket = fetch-and-add(&lockvar->nextticket, 1);  
while(1) {  
    if(lockvar->isready[myticket] == 1) break;  
}
```

UNLOCK:

```
lockvar->isready[myticket + 1] = 1;
```

What are the storage requirements?

Array-Based Queuing Lock

LOCK:

```
myticket = fetch-and-add(&lockvar->nextticket, 1);  
while(1) {  
    if(lockvar->isready[myticket] == 1) break;  
}
```

can add circular/overflow logic to bound array,
but imposes constraint on number of threads waiting

UNLOCK:

```
lockvar->isready[myticket + 1] = 1;
```

What are the storage requirements?

List-Based Queuing Lock (MCS Lock)

LOCK:

```
myticket->next = NULL;  
tail = fetch-and-store(lockvar, myticket);  
if(tail != NULL) {  
    myticket->isready = 0;  
    tail->next = myticket;  
    while(myticket->isready == 0);  
}
```

UNLOCK:

```
if(myticket->next == NULL) {  
    if(compare-and-swap(lockvar, myticket, NULL) == myticket)  
        return;  
    while(myticket->next == NULL);  
}  
myticket->next->isready = 1;
```

List-Based Queuing Lock (MCS Lock)

LOCK:

```
myticket->next = NULL;  
tail = fetch-and-store(lockvar, myticket);  
if(tail != NULL) {  
    myticket->isready = 0;  
    tail->next = myticket;  
    while(myticket->isready == 0);  
}
```

add me to tail of locker list

if lockers before me, wait

UNLOCK:

```
if(myticket->next == NULL) {  
    if(compare-and-swap(lockvar, myticket, NULL) == myticket)  
        return;  
    while(myticket->next == NULL);  
}  
myticket->next->isready = 1;
```

check if no lockers after me

double-check, if so, done

wait for new lockers to wait

wake up next locker



List-Based Queuing Lock (MCS Lock)

LOCK:

```
myticket->next = NULL;  
tail = fetch-and-store(lockvar, myticket);  
if(tail != NULL) {  
    myticket->isready = 0;  
    tail->next = myticket;  
    while(myticket->isready == 0);  
}
```

add me to tail of locker list

if lockers before me, wait

UNLOCK:

```
if(myticket->next == NULL) {  
    if(compare-and-swap(lockvar, myticket, NULL) == myticket)  
        return;  
    while(myticket->next == NULL);  
}  
myticket->next->isready = 1;
```

check if no lockers after me

double-check, if so, done

wait for new lockers to wait

wake up next locker

releasing lock needs atomic op
and blocking



List-Based Queuing Lock (MCS Lock)

LOCK:

```
myticket->next = NULL;
tail = fetch-and-store(lockvar, myticket);
if(tail != NULL) {
    myticket->isready = 0;
    tail->next = myticket;
    while(myticket->isready == 0);
}
```

add me to tail of locker list

if lockers before me, wait

spin on private nodes of linked list,
flexible storage requirements

UNLOCK:

```
if(myticket->next == NULL) {
    if(compare-and-swap(lockvar, myticket, NULL) == myticket)
        return;
    while(myticket->next == NULL);
}
myticket->next->isready = 1;
```

check if no lockers after me

double-check, if so, done

wait for new lockers to wait

wake up next locker



List-Based Queuing Lock (MCS Lock)

LOCK:

```
myticket->next = NULL;
tail = fetch-and-store(lockvar, myticket);
if(tail != NULL) {
    myticket->isready = 0;
    tail->next = myticket;
    while(myticket->isready == 0);
}
```

UNLOCK:

```
if(myticket->next == NULL) {
    if(compare-and-swap(lockvar, myticket, NULL) == myticket)
        return;
    while(myticket->next == NULL);
}
myticket->next->isready = 1;
```

lockvar: NULL

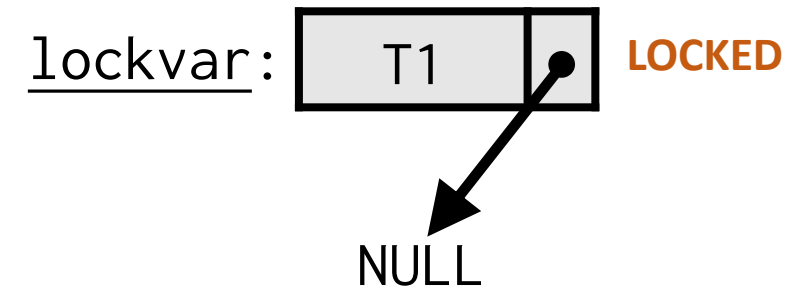
List-Based Queuing Lock (MCS Lock)

LOCK:

```
myticket->next = NULL;
tail = fetch-and-store(lockvar, myticket);
if(tail != NULL) {
    myticket->isready = 0;
    tail->next = myticket;
    while(myticket->isready == 0);
}
```

UNLOCK:

```
if(myticket->next == NULL) {
    if(compare-and-swap(lockvar, myticket, NULL) == myticket)
        return;
    while(myticket->next == NULL);
}
myticket->next->isready = 1;
```



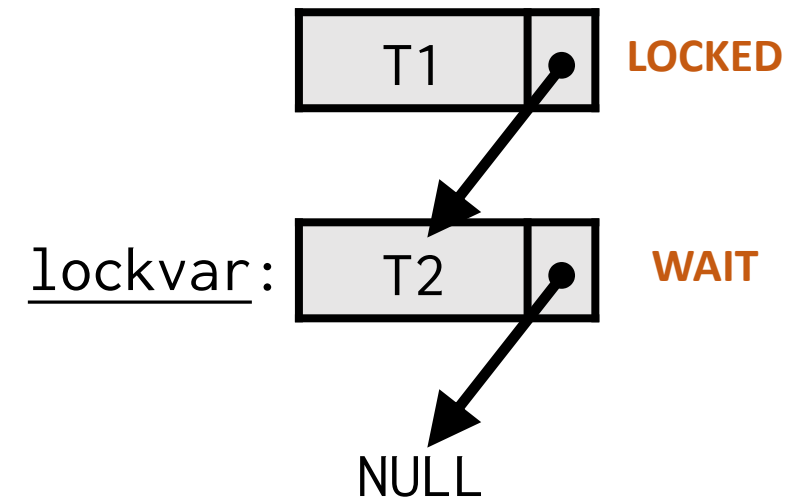
List-Based Queuing Lock (MCS Lock)

LOCK:

```
myticket->next = NULL;
tail = fetch-and-store(lockvar, myticket);
if(tail != NULL) {
    myticket->isready = 0;
    tail->next = myticket;
    while(myticket->isready == 0);
}
```

UNLOCK:

```
if(myticket->next == NULL) {
    if(compare-and-swap(lockvar, myticket, NULL) == myticket)
        return;
    while(myticket->next == NULL);
}
myticket->next->isready = 1;
```



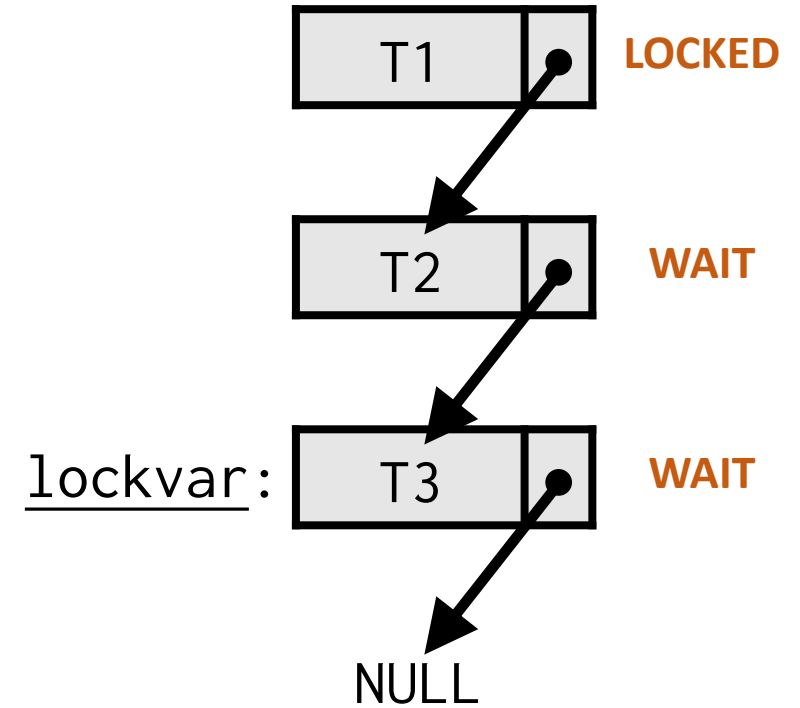
List-Based Queuing Lock (MCS Lock)

LOCK:

```
myticket->next = NULL;
tail = fetch-and-store(lockvar, myticket);
if(tail != NULL) {
    myticket->isready = 0;
    tail->next = myticket;
    while(myticket->isready == 0);
}
```

UNLOCK:

```
if(myticket->next == NULL) {
    if(compare-and-swap(lockvar, myticket, NULL) == myticket)
        return;
    while(myticket->next == NULL);
}
myticket->next->isready = 1;
```



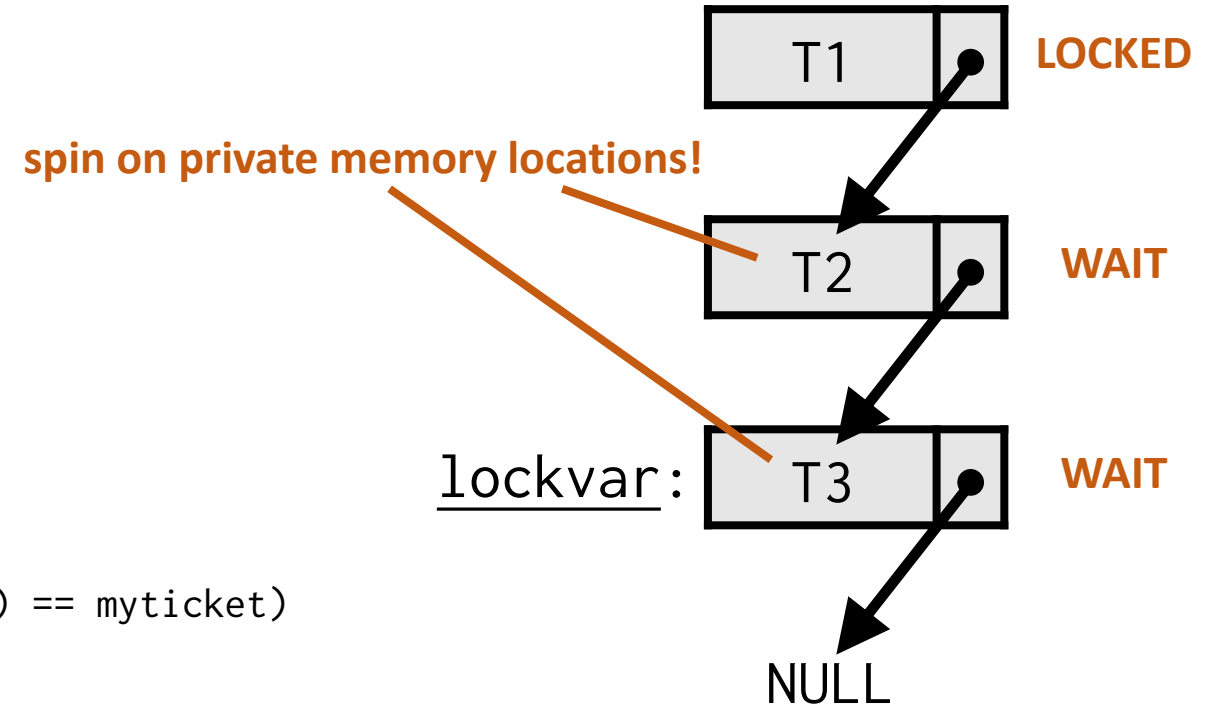
List-Based Queuing Lock (MCS Lock)

LOCK:

```
myticket->next = NULL;
tail = fetch-and-store(lockvar, myticket);
if(tail != NULL) {
    myticket->isready = 0;
    tail->next = myticket;
    while(myticket->isready == 0);
}
```

UNLOCK:

```
if(myticket->next == NULL) {
    if(compare-and-swap(lockvar, myticket, NULL) == myticket)
        return;
    while(myticket->next == NULL);
}
myticket->next->isready = 1;
```



List-Based Queuing Lock (MCS Lock)

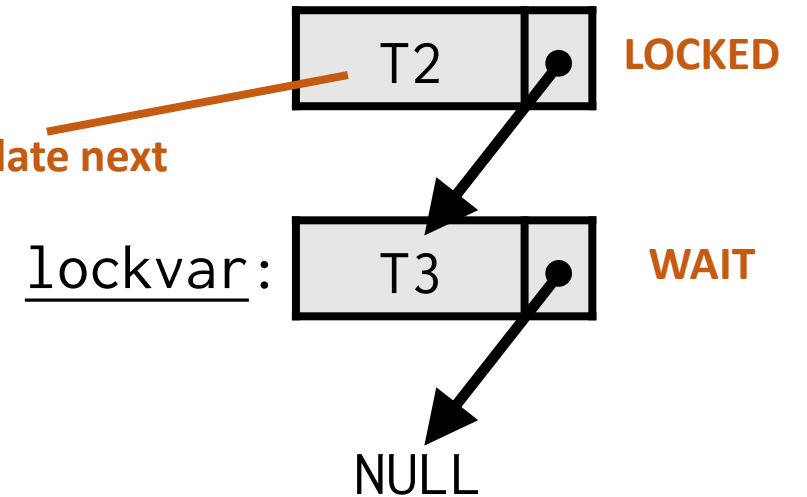
LOCK:

```
myticket->next = NULL;
tail = fetch-and-store(lockvar, myticket);
if(tail != NULL) {
    myticket->isready = 0;
    tail->next = myticket;
    while(myticket->isready == 0);
}
```

UNLOCK:

```
if(myticket->next == NULL) {
    if(compare-and-swap(lockvar, myticket, NULL) == myticket)
        return;
    while(myticket->next == NULL);
}
myticket->next->isready = 1;
```

FIFO order and only invalidate next



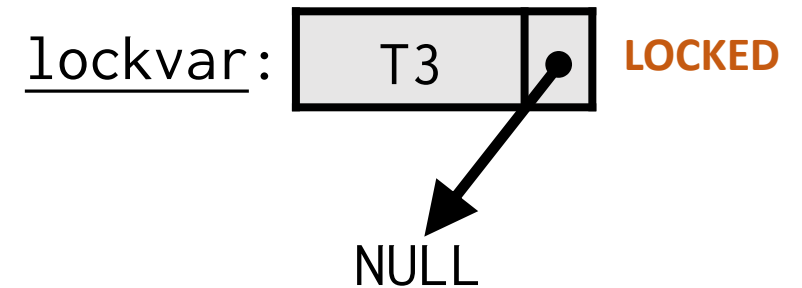
List-Based Queuing Lock (MCS Lock)

LOCK:

```
myticket->next = NULL;
tail = fetch-and-store(lockvar, myticket);
if(tail != NULL) {
    myticket->isready = 0;
    tail->next = myticket;
    while(myticket->isready == 0);
}
```

UNLOCK:

```
if(myticket->next == NULL) {
    if(compare-and-swap(lockvar, myticket, NULL) == myticket)
        return;
    while(myticket->next == NULL);
}
myticket->next->isready = 1;
```



List-Based Queuing Lock (MCS Lock)

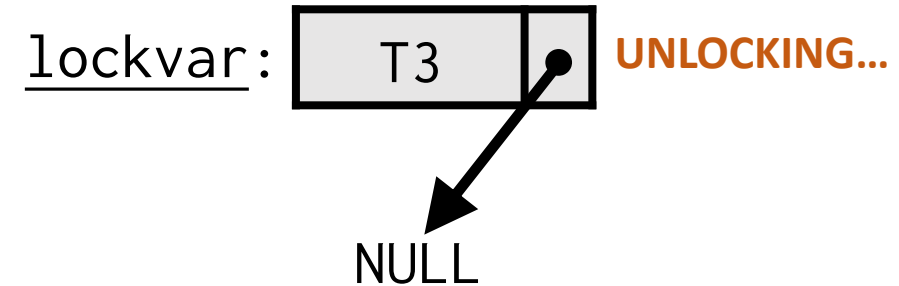
LOCK:

```
myticket->next = NULL;
tail = fetch-and-store(lockvar, myticket);
if(tail != NULL) {
    myticket->isready = 0;
    tail->next = myticket;
    while(myticket->isready == 0);
}
```

UNLOCK:

```
if(myticket->next == NULL) {
    if(compare-and-swap(lockvar, myticket, NULL) == myticket)
        return;
    while(myticket->next == NULL);
}
myticket->next->isready = 1;
```

T3



List-Based Queuing Lock (MCS Lock)

LOCK:

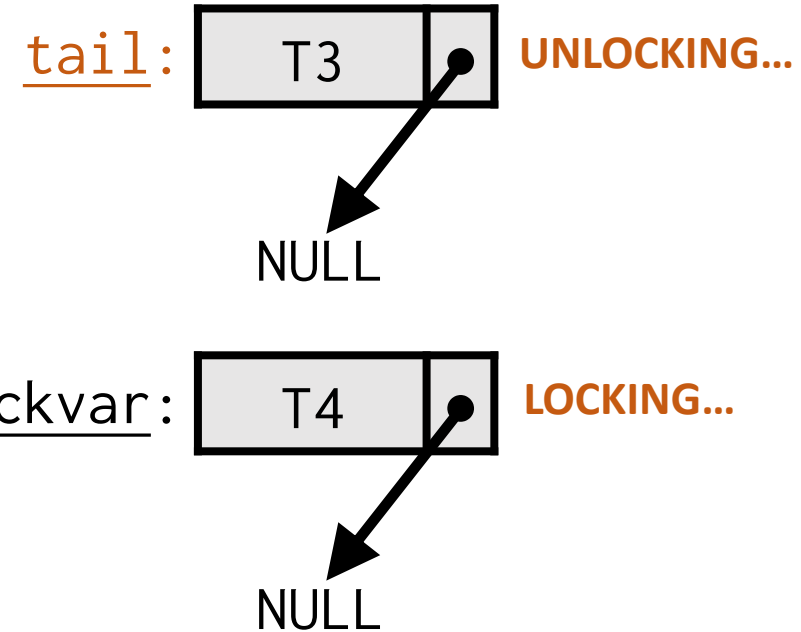
```
myticket->next = NULL;  
tail = fetch-and-store(lockvar, myticket);  
if(tail != NULL) {  
    myticket->isready = 0;  
    tail->next = myticket;  
    while(myticket->isready == 0);  
}
```

T4

UNLOCK:

```
if(myticket->next == NULL) {  
    if(compare-and-swap(lockvar, myticket, NULL) == myticket)  
        return;  
    while(myticket->next == NULL);  
}  
myticket->next->isready = 1;
```

T3



List-Based Queuing Lock (MCS Lock)

LOCK:

```
myticket->next = NULL;  
tail = fetch-and-store(lockvar, myticket);  
if(tail != NULL) {  
    myticket->isready = 0;  
    tail->next = myticket;  
    while(myticket->isready == 0);  
}
```

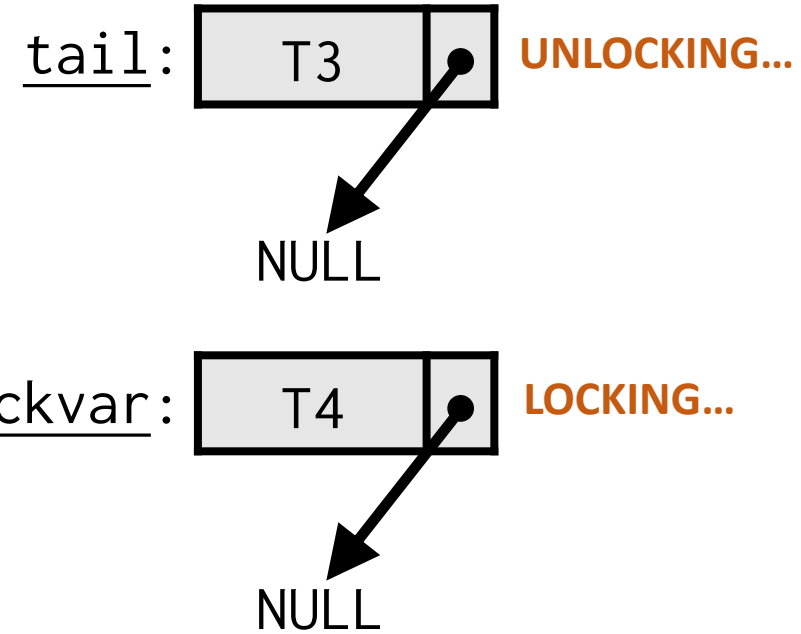
T4

next is NULL,
but not done yet...

T3

UNLOCK:

```
if(myticket->next == NULL) {  
    if(compare-and-swap(lockvar, myticket, NULL) == myticket)  
        return;  
    while(myticket->next == NULL);  
}  
myticket->next->isready = 1;
```



List-Based Queuing Lock (MCS Lock)

LOCK:

```
myticket->next = NULL;  
tail = fetch-and-store(lockvar, myticket);  
if(tail != NULL) {  
    myticket->isready = 0;  
    tail->next = myticket;  
    while(myticket->isready == 0);  
}
```

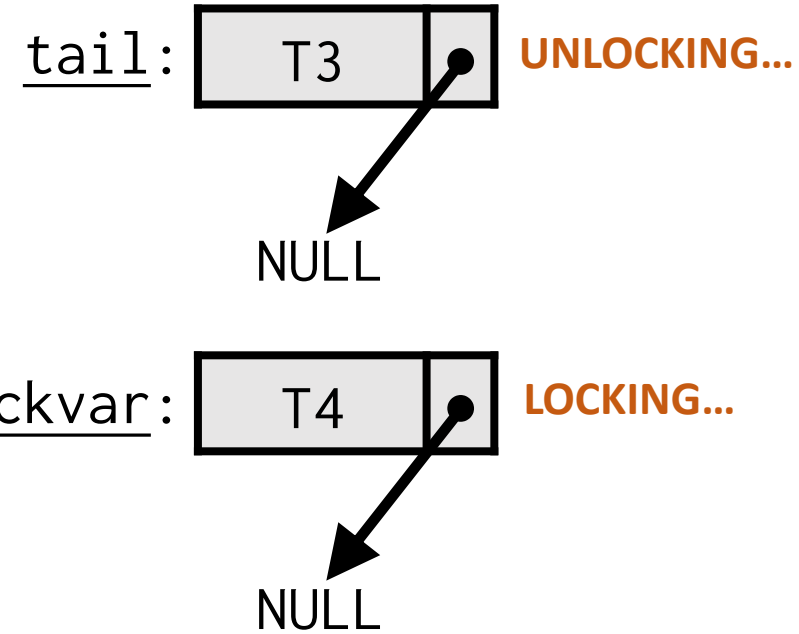
T4

UNLOCK:

```
if(myticket->next == NULL) {  
    if(compare-and-swap(lockvar, myticket, NULL) == myticket)  
        return;  
    while(myticket->next == NULL);  
}  
myticket->next->isready = 1;
```

T3

...double-check: fail...



List-Based Queuing Lock (MCS Lock)

LOCK:

```
myticket->next = NULL;  
tail = fetch-and-store(lockvar, myticket);  
if(tail != NULL) {  
    myticket->isready = 0;  
    tail->next = myticket;  
    while(myticket->isready == 0);  
}
```

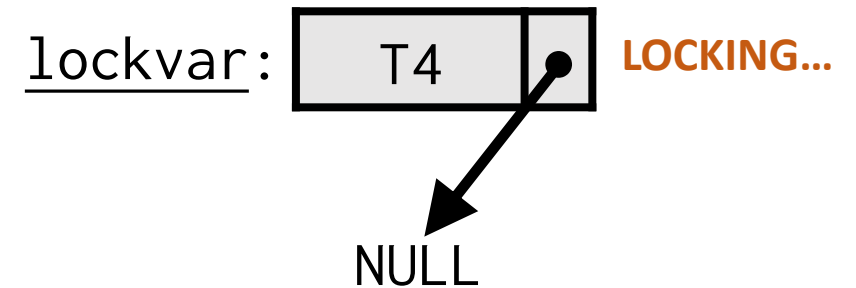
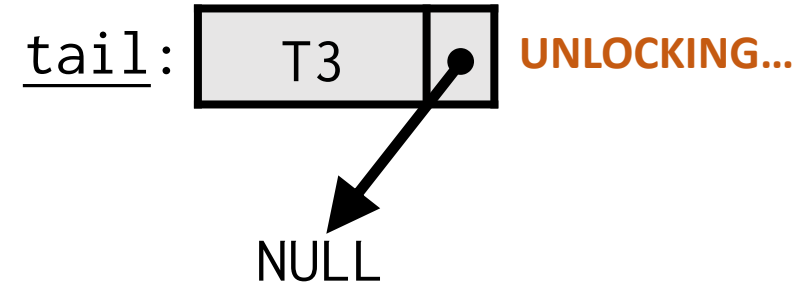
T4

UNLOCK:

```
if(myticket->next == NULL) {  
    if(compare-and-swap(lockvar, myticket, NULL) == myticket)  
        return;  
    while(myticket->next == NULL);  
}
```

T3

...wait



List-Based Queuing Lock (MCS Lock)

LOCK:

```
myticket->next = NULL;
tail = fetch-and-store(lockvar, myticket);
if(tail != NULL) {
    myticket->isready = 0;
    tail->next = myticket;
    while(myticket->isready == 0);
}
```

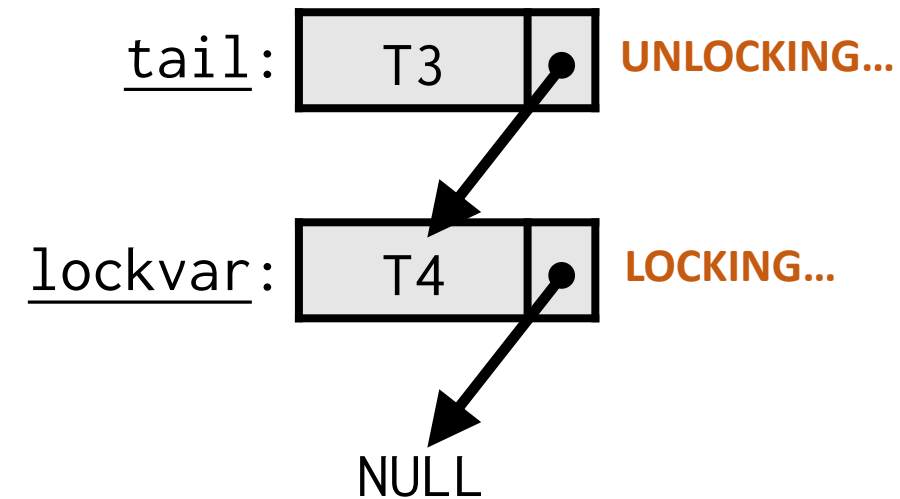
T4

UNLOCK:

```
if(myticket->next == NULL) {
    if(compare-and-swap(lockvar, myticket, NULL) == myticket)
        return;
    while(myticket->next == NULL);
}
```

T3

unblocked!



List-Based Queuing Lock (MCS Lock)

LOCK:

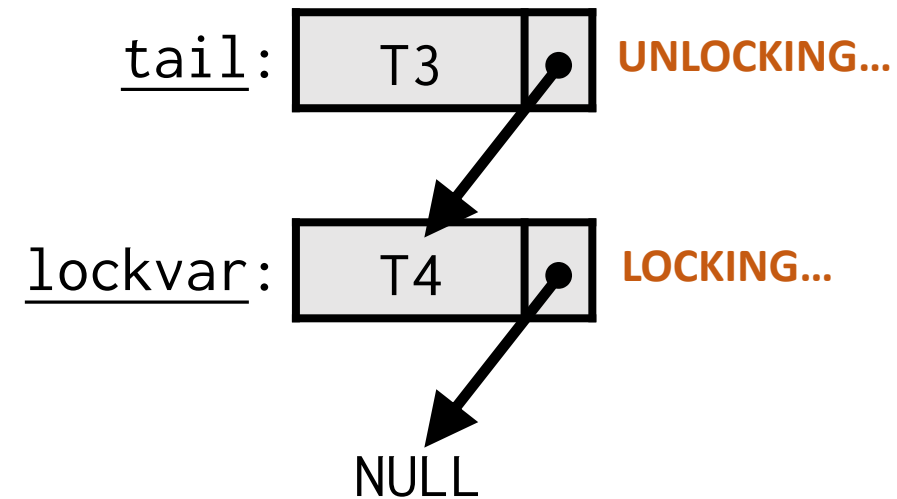
```
myticket->next = NULL;  
tail = fetch-and-store(lockvar, myticket);  
if(tail != NULL) {  
    myticket->isready = 0;  
    tail->next = myticket;  
    while(myticket->isready == 0);  
}
```

T4 **wait**

UNLOCK:

```
if(myticket->next == NULL) {  
    if(compare-and-swap(lockvar, myticket, NULL) == myticket)  
        return;  
    while(myticket->next == NULL);  
}  
myticket->next->isready = 1;
```

T3 **unblocked!**



List-Based Queuing Lock (MCS Lock)

LOCK:

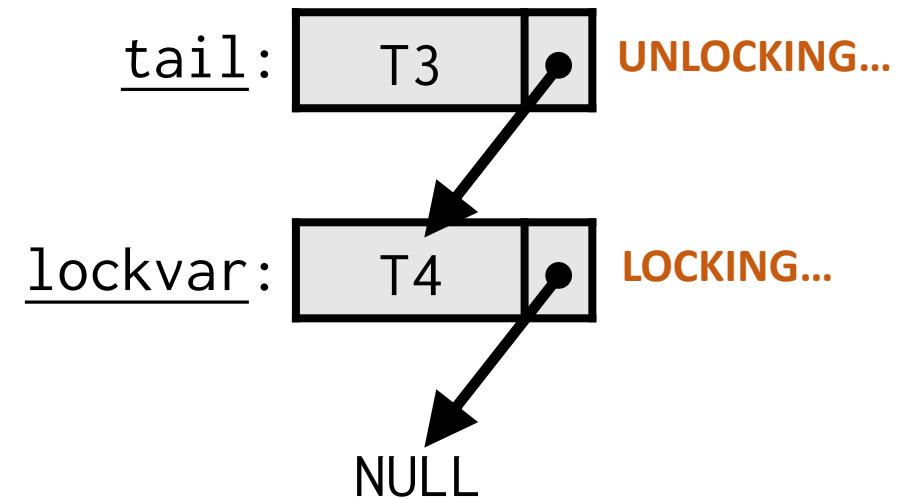
```
myticket->next = NULL;
tail = fetch-and-store(lockvar, myticket);
if(tail != NULL) {
    myticket->isready = 0;
    tail->next = myticket;
    while(myticket->isready == 0);
}
```

T4 **unblocked!**

UNLOCK:

```
if(myticket->next == NULL) {
    if(compare-and-swap(lockvar, myticket, NULL) == myticket)
        return;
    while(myticket->next == NULL);
}
```

T3



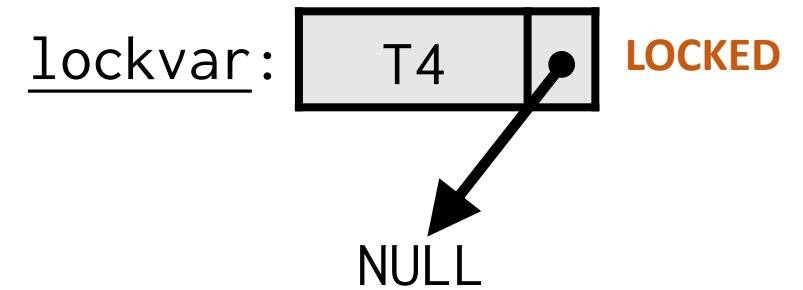
List-Based Queuing Lock (MCS Lock)

LOCK:

```
myticket->next = NULL;
tail = fetch-and-store(lockvar, myticket);
if(tail != NULL) {
    myticket->isready = 0;
    tail->next = myticket;
    while(myticket->isready == 0);
}
```

UNLOCK:

```
if(myticket->next == NULL) {
    if(compare-and-swap(lockvar, myticket, NULL) == myticket)
        return;
    while(myticket->next == NULL);
}
myticket->next->isready = 1;
```



List-Based Queuing Lock (MCS Lock)

LOCK:

```
myticket->next = NULL;  
tail = fetch-and-store(lockvar, myticket);  
if(tail != NULL) {  
    myticket->isready = 0;  
    tail->next = myticket;  
    while(myticket->isready == 0);  
}
```

can we use fetch-and-add instead?

UNLOCK:

```
if(myticket->next == NULL) {  
    if(compare-and-swap(lockvar, myticket, NULL) == myticket)  
        return;  
    while(myticket->next == NULL);  
}  
myticket->next->isready = 1;
```

List-Based Queuing Lock (MCS Lock)

LOCK:

```
myticket->next = NULL;
while(1) {
    tmp = *lockvar;
    tail = fetch-and-add(lockvar, myticket - tmp);
    if(tail == tmp) break;
}
if(tail != NULL) {
    myticket->isready = 0;
    tail->next = myticket;
    while(myticket->isready == 0);
}
```

UNLOCK:

```
if(myticket->next == NULL) {
    if(compare-and-swap(lockvar, myticket, NULL) == myticket)
        return;
    while(myticket->next == NULL);
}
myticket->next->isready = 1;
```

List-Based Queuing Lock (MCS Lock)

LOCK:

does this actually work?

```
myticket->next = NULL;
while(1) {
    tmp = *lockvar;
    tail = fetch-and-add(lockvar, myticket - tmp);
    if(tail == tmp) break;
}
if(tail != NULL) {
    myticket->isready = 0;
    tail->next = myticket;
    while(myticket->isready == 0);
}
```

UNLOCK:

```
if(myticket->next == NULL) {
    if(compare-and-swap(lockvar, myticket, NULL) == myticket)
        return;
    while(myticket->next == NULL);
}
myticket->next->isready = 1;
```



List-Based Queuing Lock (MCS Lock)

LOCK:

```
myticket->next = NULL;  
tail = fetch-and-store(lockvar, myticket);  
if(tail != NULL) {  
    myticket->isready = 0;  
    tail->next = myticket;  
    while(myticket->isready == 0);  
}
```

can we use compare-and-swap instead?

UNLOCK:

```
if(myticket->next == NULL) {  
    if(compare-and-swap(lockvar, myticket, NULL) == myticket)  
        return;  
    while(myticket->next == NULL);  
}  
myticket->next->isready = 1;
```

List-Based Queuing Lock (MCS Lock)

LOCK:

```
myticket->next = NULL;
while(1) {
    tail = *lockvar;
    if(compare-and-swap(lockvar, tail, myticket) == tail)
        break;
}
if(tail != NULL) {
    myticket->isready = 0;
    tail->next = myticket;
    while(myticket->isready == 0);
}
```

UNLOCK:

```
if(myticket->next == NULL) {
    if(compare-and-swap(lockvar, myticket, NULL) == myticket)
        return;
    while(myticket->next == NULL);
}
myticket->next->isready = 1;
```

Coarse-Grained Locking

lock

... = **A**

...

A = ...

unlock

lock

... = **A**

...

A = ...

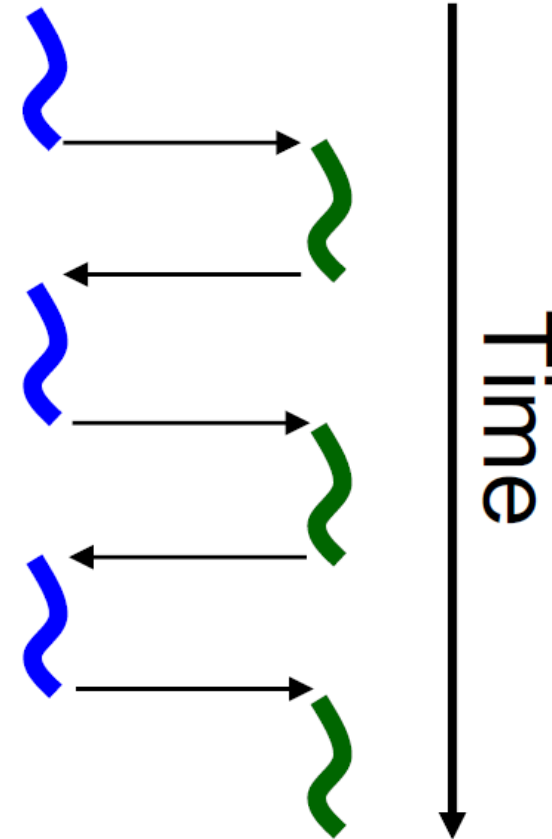
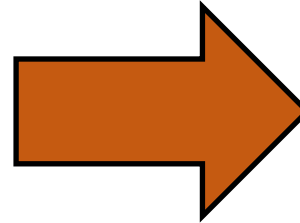
unlock



Coarse-Grained Locking

lock
... = **A**
...
A = ...
unlock

lock
... = **A**
...
A = ...
unlock

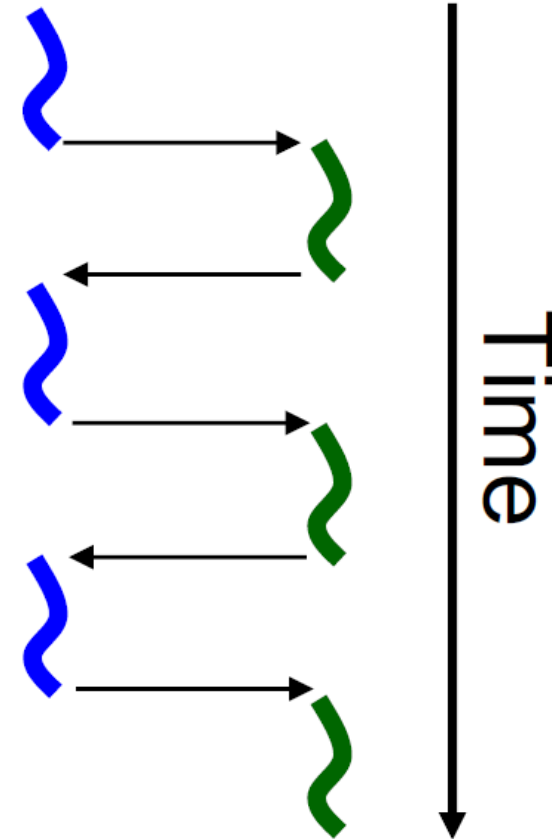
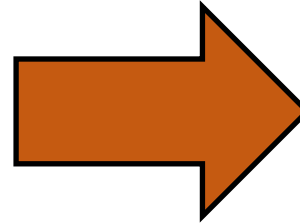


Coarse-Grained Locking

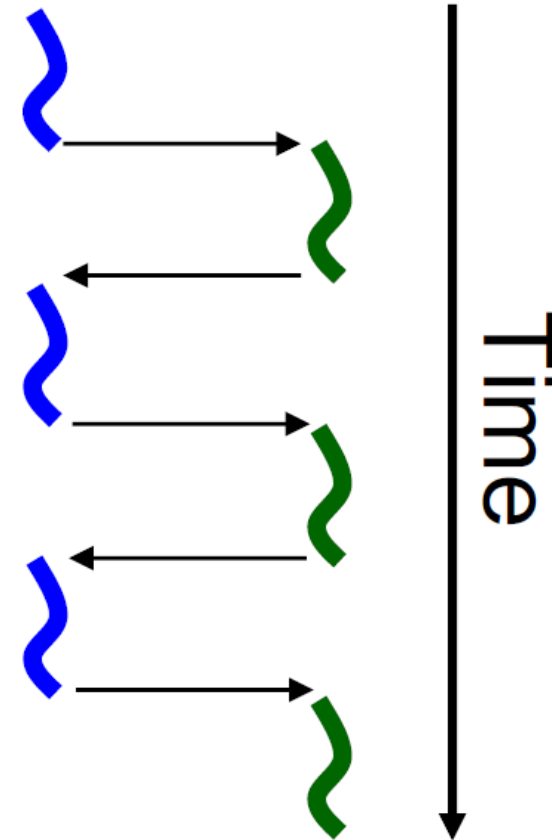
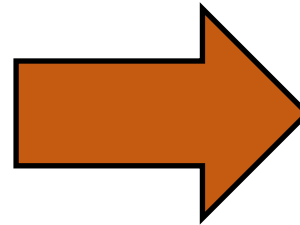
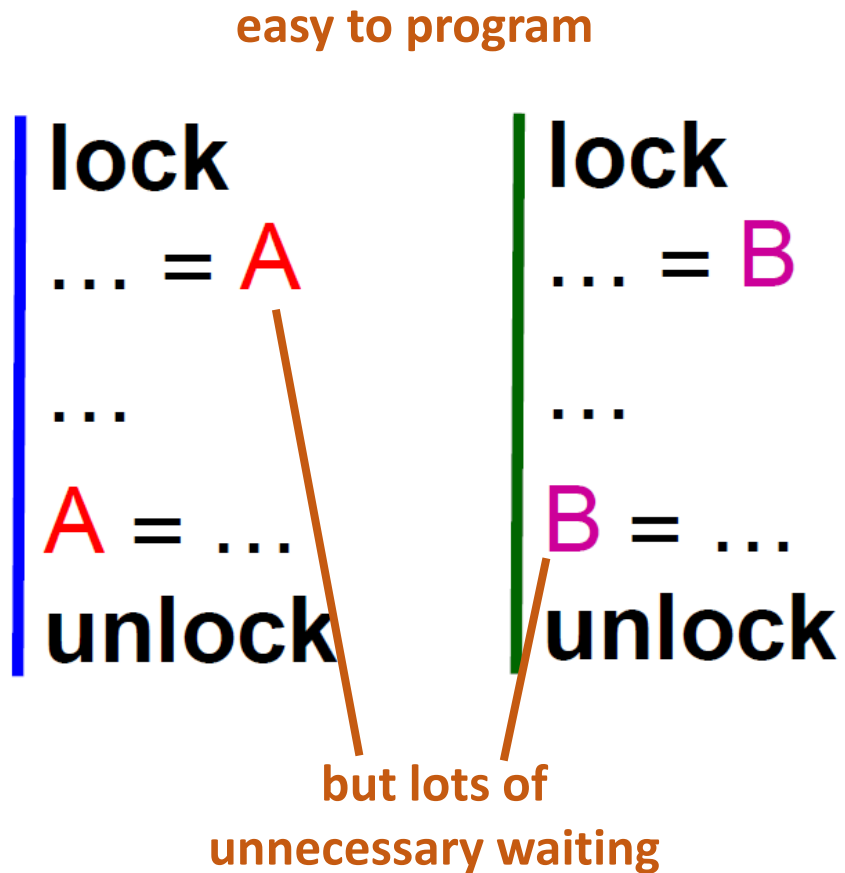
easy to program

```
lock  
... = A  
...  
A = ...  
unlock
```

```
lock  
... = A  
...  
A = ...  
unlock
```



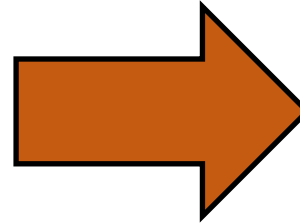
Coarse-Grained Locking



Fine-Grained Locking

less waiting

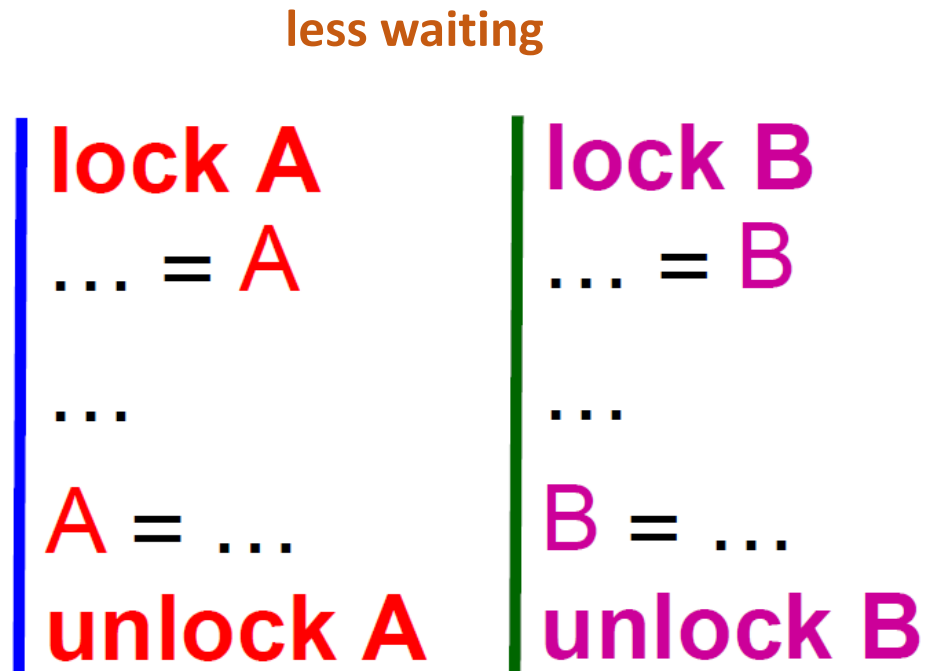
lock A	lock B
... = A	... = B
...	...
A = ...	B = ...
unlock A	unlock B



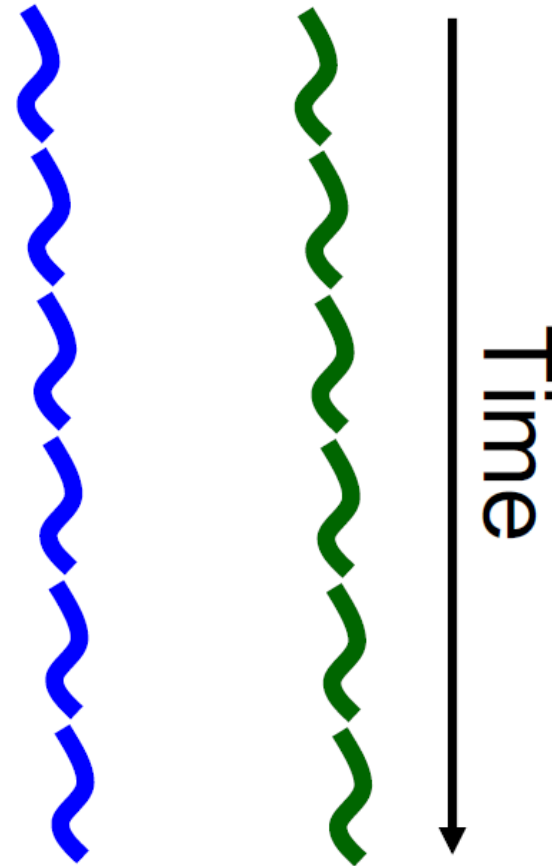
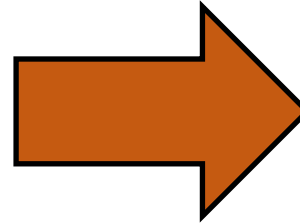
Time



Fine-Grained Locking



but harder to program...



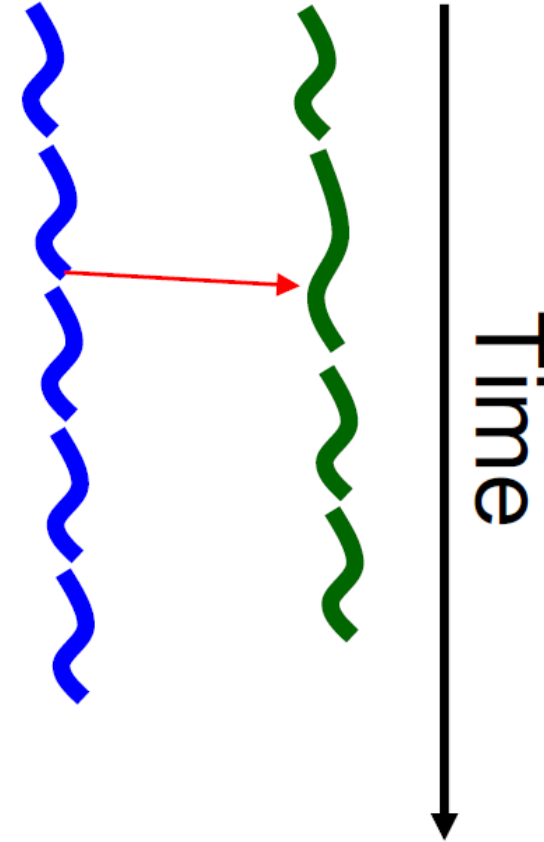
Fine-Grained Locking

lock A	lock B
... = A	... = B
...	...
if (...){	if (...){
lock B	lock A
B = ...	A = ...
unlock B	unlock A
}	}
...	...
A = ...	B = ...
unlock A	unlock B



Fine-Grained Locking

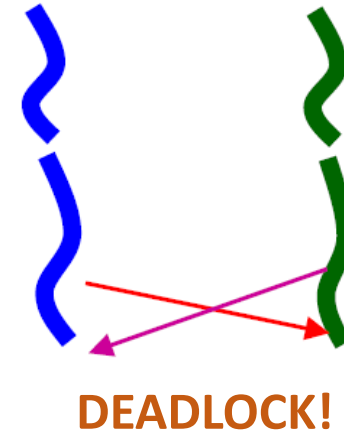
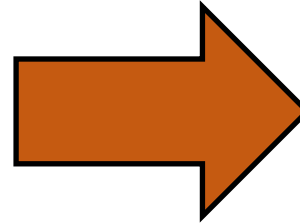
lock A	lock B
... = A	... = B
...	...
if (...){	if (...){
lock B	lock A
B = ...	A = ...
unlock B	unlock A
}	}
...	...
A = ...	B = ...
unlock A	unlock B



Fine-Grained Locking

lock A	lock B
... = A	... = B
...	...
if (...){	if (...){
lock B	lock A
B = ...	A = ...
unlock B	unlock A
}	}
...	...
A = ...	B = ...
unlock A	unlock B

if unlucky...



Time



Deadlock Avoidance

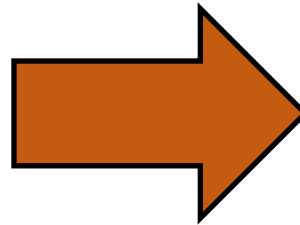
lock A	lock B
... = A	... = B
...	...
if (...){	if (...){
lock B	lock A
B = ...	A = ...
unlock B	unlock A
}	}
...	...
A = ...	B = ...
unlock A	unlock B



Deadlock Avoidance

lock A	lock B
... = A	... = B
...	...
if (...){	if (...){
lock B	lock A
B = ...	A = ...
unlock B	unlock A
}	}
...	...
A = ...	B = ...
unlock A	unlock B

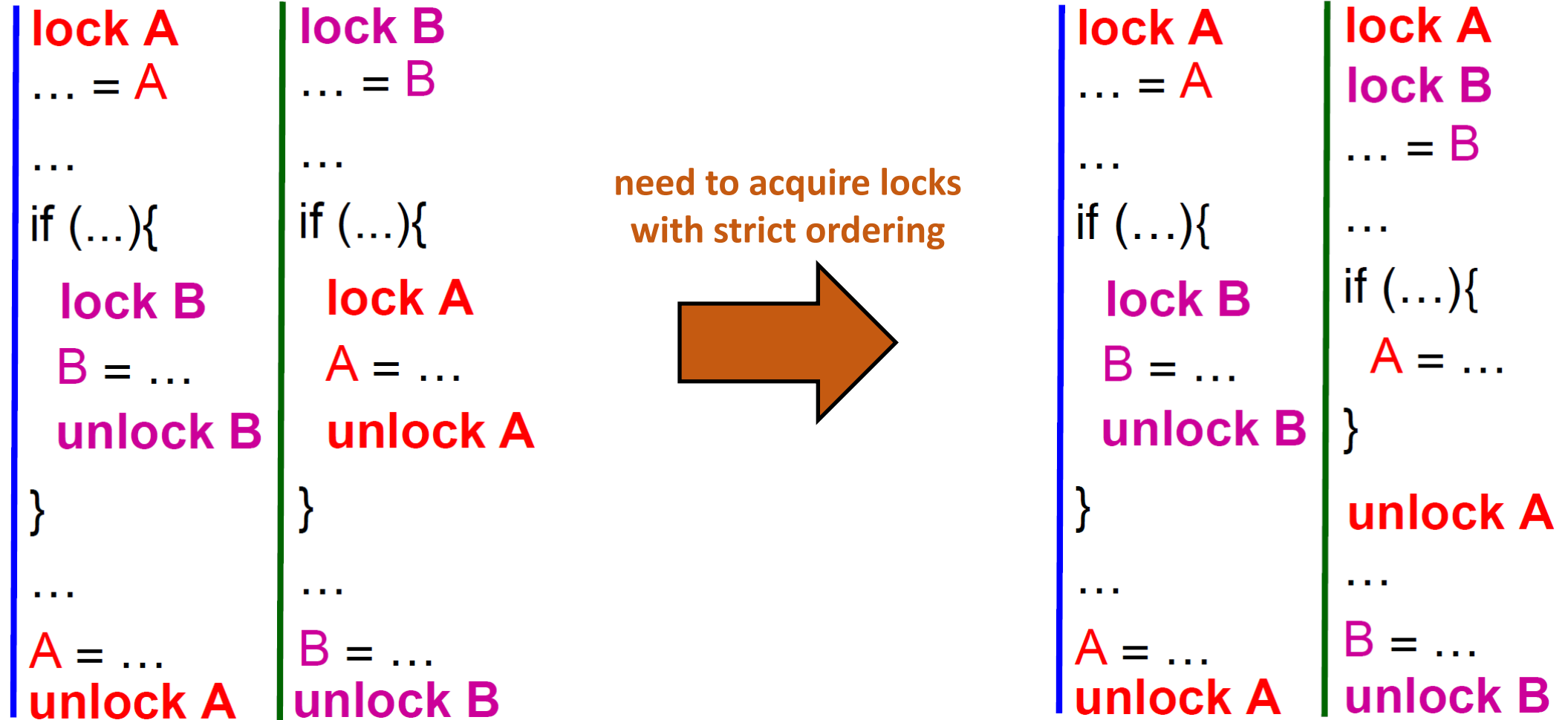
need to acquire locks
with strict ordering



lock A	lock A
... = A	lock B
...	... = B
...	...
if (...){	...
lock B	if (...){
B = ...	A = ...
unlock B	}
}	unlock A
...	...
A = ...	B = ...
unlock A	unlock B



Deadlock Avoidance



Do locks need to be released in order too?